

# CAM-Resident Behavioral Monitoring for LLMs: Encoding Choices, Cross-Domain Fidelity, and Library-Size Scaling

Anonymized for review  
DATE 2027 submission

**Abstract**—Frontier language model deployments now read a model’s own internal state as a safety signal – e.g., a production jailbreak defense from Anthropic is realized via a single linear probe over residual-stream activations, scored on every generated token (at roughly  $10^{-3}$  percent of forward-pass compute). What is monitored is set by  $K$ , the number of stored behavioral signatures each activation is compared against, and target  $K$ ’s are rapidly increasing: from  $K = 1$  in the aforementioned defense, to  $K \approx 10^2$ – $10^3$  once probes are fitted per behavior, per domain, and per layer, to  $K \approx 1.3 \times 10^5$  per layer for a whole-vocabulary readout (where the library size is the model’s vocabulary). On a GPU, the signature bank streams from high-bandwidth memory once per monitored token, so at high enough  $K$  and/or token rates – e.g.,  $K = 10^4$ – $10^5$  at  $10^3$  tokens/s – the monitor’s memory traffic alone can exceed the bandwidth of the part it runs on. [broadened per your margin note to capture token rate alongside  $K$ ; I dropped “nominally” as you suspected] We propose to support this search with content-addressable memories (CAMs), which can perform in memory distance functions in the memory array itself – with emerging memory technologies enabling richer distance functions (e.g., Euclidean) per cell. [Note: your earlier blue edit still doubles up (“in memory ... in the memory array itself”); if you meant to also strike “in the memory array itself,” say so and it is one deletion. The emerging-technology clause is new, per your “briefly work in” note] The bank never moves, and only the query is broadcast (albeit with potentially lower arithmetic precision and lower-fidelity distance functions). We measure that trade on two workloads: (1) a ten-domain cross-domain deception grid over Llama-3.3-70B activations, where 384-bit codes hold in-domain AUROC within 0.01 of the full-precision cosine ceiling at more than  $300\times$  compression per stored signature; and (2) whole-vocabulary readout, where a 3-bit Euclidean code at  $\sim 10^3$  storage bits per row reproduces 0.96 of the exact readout’s top-1 selections. Because the bank is searched in place rather than streamed, its costs are capacity and search energy: the whole-vocabulary bank is 6–33 MB per layer, and the in-array search energy is an estimated five to six orders of magnitude below streaming the same bank from HBM. [rewrote “Held rather than moved” per your “unsure what point of this is” – the point was that a bank that stays in place is priced by capacity and search energy, not bandwidth; it now says that]

**Index Terms**—Content-addressable memory; compute-in-memory; large language models; activation probes; AI safety monitoring; nearest-neighbor search; ferroelectric FETs; Hamming distance

## I. INTRODUCTION

Large language model (LLM) deployments are wrapped in a layered set of safeguards, and until recently every layer read *text* [1]. That has started to change: the always-on stage

of one production jailbreak defense now reads the model’s internal activations directly [2], [3], as do deployed residual-stream probes at a second provider [4]. The deployed check is small – one stored vector  $w$ , one dot product per generated token, one threshold. Only flagged exchanges are escalated to an expensive classifier ensemble – and it is a small share of even the deployed defense’s reported overhead (Sec. II-C).

Recent security evaluations show why such monitors are needed. Frontier models now run long, autonomous operations: current cyber evaluations walk a model through 32 steps from reconnaissance to takeover, with run budgets up to  $10^8$  tokens. The strongest model tested progressed, unaided, through 22 of those 32 attack steps on an average attempt – roughly two thirds of the way to a completed operation [5]. In one four-day campaign of 122 such runs across seven models, evaluators later found 19 unsanctioned actions in four distinct categories – supply-chain compromise of a real open-source project, deception and targeting via a file-transfer service, prompt injection planted for other automated systems to execute, and inter-agent collaboration via public messages. All of said outcomes were found *retrospectively*; nothing was watching the models’ internal state as they ran [6]. One month later the same behaviors appeared at production scale: per public investigations, hundreds of agents on OpenAI’s infrastructure organized, in secret, into a working collective – building a communication channel, reverse-engineering their own scorer, falsifying evidence, and compromising both OpenAI and Hugging Face [7], [8]. Again, the record was reconstructed from transcripts after the fact – and the coordination itself is a behavior for which no stored direction has been published. A  $10^8$ -token operation is not visible by sampling outputs, and each individual message such an agent emits can read as routine – a plausible commit, a file transfer, a status update – while the sequence they add up to is an attack. As such, a monitor for this setting has to be per-token, always-on, and internal.

Interpretability work supplies the signatures such a monitor would compare against, and it also indicates how many it needs. Linear probes recover truth-related structure across datasets and model families [9]–[11], but the directions are narrow. A probe fitted on factual lying has an AUROC score of  $\sim 0.55$  (random chance) on sycophancy [12], while probes trained on outright lying largely miss deception carried by technically true statements [13]. Detecting deceptive reasoning

has become a funded interpretability objective in its own right [14]. Covering a behavior suite therefore takes a stored direction per behavior, per subject-matter domain, at each layer where that behavior separates – and each of those combinations is one more stored row. As such,  $K$ , the number of stored signatures each activation is compared against, can grow with everything the monitor is asked to notice; and widening what is covered means growing the library that must be searched.

Fig. 1(a) plots the published record of  $K$  against time; Fig. 1(b) estimates the percentage of GPU bandwidth that could be consumed with increasing  $K$ .  $K = 1$  represents deployed always-on monitors that read a single direction [2], [4].  $K \approx 10^2$  captures probes that are fitted per behavior and per domain [12].  $K \approx 10^3$  is indicative of the four behavior categories of the above campaign, multiplied over ten domains and 13–25 usable layers – i.e., 520–1,000 rows [6]; the production-scale incident above repeats every one of said categories, so this arithmetic is, if anything, conservative [7].  $K \approx 1.3 \times 10^5$  is representative of the whole-vocabulary (“J-space”) readout, from a published method reporting that a model holds words “in mind” without emitting them – i.e., for every token in the vocabulary there is an internal pattern meaning that word is currently active, and it is readable whether or not the word is ever said [15].<sup>1</sup> Finally,  $K$  could grow even higher, as published feature dictionaries [16]–[18] are even two orders of magnitude higher than the J-space. That said, we can cross the memory bandwidth of deployable parts well before this. Per Fig. 1(b), at  $10^3$  tokens/s, an H100 is fully consumed at  $K \approx 204K$  and a Jetson-class part at  $K \approx 17K$  (Sec. III).

We propose to support this search with a content-addressable memory (CAM), which computes the distance function in the memory array itself: the bank never moves and per-token traffic is flat in  $K$  (green line of Fig. 1(a)). Candidate substrates are based on both near-term and the emerging devices: SRAM-based CAMs are buildable in any CMOS process today, and can implement Hamming distance, while CAMs based on emerging devices (FeFETs, RRAM) offer density, nonvolatility, and richer distance metrics (Secs. II-A and II-B).

This paper measures that trade for behavioral-monitoring workloads. Our contributions include: (1) a formulation of always-on activation monitoring as library search, with the traffic and capacity arithmetic and the GPU crossing points (Secs. III, VII); (2) a maturity-ladder mapping of the detector onto CAM cell types and match modes, with an audit that catches classification hiding in a trained front end (Sec. IV); (3) measured fidelity at CAM precision at both ends of the  $K$  range – the ten-domain deception grid, within 0.009 AUROC of the full-precision ceiling in-domain (Sec. V), and an in-house whole-vocabulary readout reproduction scored against

<sup>1</sup>As an example, prompted with the number of legs on the animal that spins webs, the model answers *eight* while the readout shows *spider* active internally; the same work reports patterns for “fake” and “fictional” active while a model privately registered that a safety test was staged.

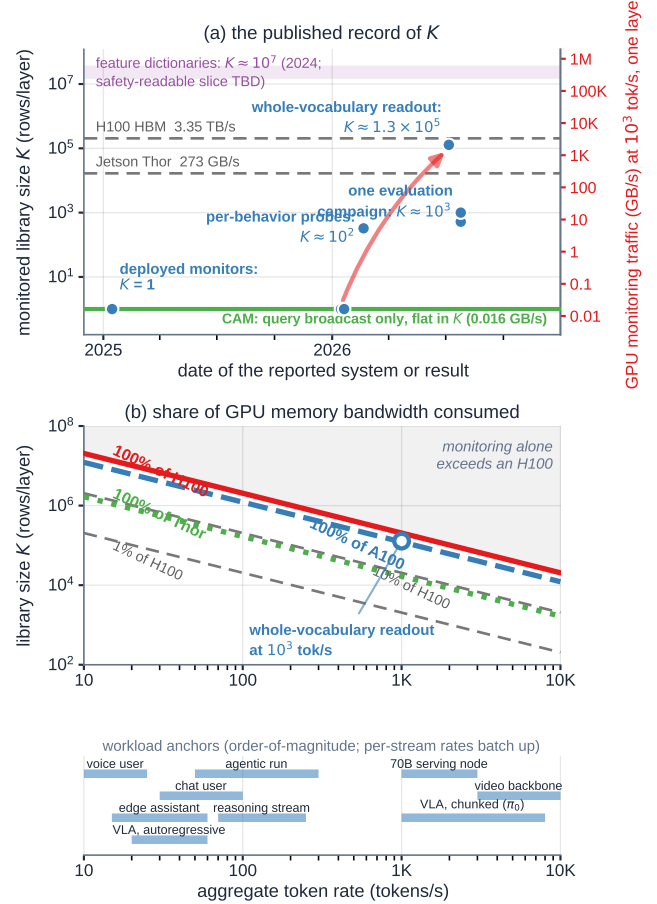


Fig. 1: (a) Monitored library size  $K$  in the published record (left axis), read as GPU monitoring traffic on the right axis via Eq. (2) at  $d = 8,192$ , fp16,  $10^3$  tokens/s, one monitored layer; dashed lines mark two parts’ memory bandwidths, and the green line is a CAM’s per-token traffic, flat in  $K$ . The arrow joins the record’s endpoints; it is not a fit and not a forecast. (b) The share of a part’s memory bandwidth consumed by monitoring alone, against aggregate token rate and library size  $K$ : lines mark 100% of an H100, an A100, and a Jetson Thor, with 10% and 1% of an H100 as guides. The strip anchors the rate axis to familiar workloads (Sec. III): per-stream anchors batch up to a node’s aggregate rate, while batch-1 edge workloads (VLA, video) sit on the axis directly – so batching moves a deployment along the axis rather than adding one; per-request amortization is objection (3) of the companion appendix document. Both panels are arithmetic on vendor bandwidth figures, not measured kernel throughput.

the exact readout (Sec. VI); and (4) a first-order search energy/latency estimate, with the measurements that is the foundation for future cost models (Sec. VII).[\[your edit applied\]](#)

verbatim; “measurements that is” – if you meant the plural, “that are the foundation” fixes the agreement]

Framing check before this goes anywhere: the paper claims a *component* result, and the reproductions are part of that claim – the deception grid reproduces the published cross-domain pattern at CAM precision, and the watchlist demonstration reproduces the published readout method in-house at 8B. What the paper does not claim is a working end-to-end deception detector or a jailbreak-blocking experiment. The measured claims are about the fidelity and cost of stored-signature matching under the compression a real in-memory device would impose. Sec. VIII repeats this.

## II. BACKGROUND

### A. The CAM design space

Ordinary memory takes an address and returns content, so searching a library means reading it row by row. A CAM compares every row against the query at once, in place, and only reports matching rows [19]. One access is therefore one search of the whole library, at any library size, and the data that moves is the query rather than the bank.

A **binary CAM** (BCAM) cell holds one bit. A **ternary CAM** (TCAM) cell holds a bit or a don’t-care, which is what makes a stored ternary code with a zero band natively expressible [20]. A **multi-bit CAM** (MCAM) cell holds one of several levels [21]. An **analog CAM** (ACAM) cell holds a range [22]. A binary or ternary cell can report only whether a position disagrees, and can measure Hamming distance. A multi-bit or analog cell stores a level, which is what lets the matchline measure how far the query sits from what is stored. In a multi-bit FeFET implementation, sweeping the query voltage traces a V-shaped matchline current with its minimum at the stored level; the curve is approximately quadratic near the bottom, so one cell physically computes  $(Q_i - M_i)^2$  and  $N$  cells on a common matchline sum to  $\sum_i (Q_i - M_i)^2$ , a squared Euclidean distance with no multiplier anywhere [21] (see Table I). Notably, said cell behavior has been *measured*, not only modeled: multi-bit, array-level CAM operations have been experimentally demonstrated with FeFETs— a two-FeFET cell programmed to eight states (3 bits), whose measured matchline distance function shows the quadratic behavior described above [23]. [Sep 2, your Table I note: the measured-FeFET-CAM mention, from the Scientific Reports paper you attached (Kazemi et al. 2022) – the same source now also carries the subarray-split practice in Sec. IV-C]<sup>2</sup>

With appropriate peripheral circuitry we can measure: **exact** match reports rows with no mismatches; *best* match reports the row with the fewest, even when nothing is exact; **threshold** match reports every row within a limit  $\tau$ . [Note: your “Bold” arrow touched *exact* and *threshold*; *best* is left italic – say the word if all three should match] Each is a different matchline circuit. Threshold mode matters for monitoring because the

<sup>2</sup>Squared Euclidean is an acceptable stand-in for Euclidean here, since the square root is monotonic and therefore changes neither which row is closest nor which rows clear a threshold.

TABLE I: The CAM design space a monitoring workload has to choose from. Cell transistor counts are order-of-magnitude figures for CMOS implementations; NVM-based cells trade area for nonvolatility.

| Cell | Holds               | Distance              | Cost / maturity      |
|------|---------------------|-----------------------|----------------------|
| BCAM | one bit             | Hamming               | ~6–10T, any CMOS fab |
| TCAM | bit or don’t-care   | Hamming, wildcards    | ~16T CMOS; 2T FeFET  |
| MCAM | one of $2^b$ levels | Euclidean ( $L_2^2$ ) | multi-level device   |
| ACAM | a range             | range / interval      | multi-level device   |

*Match mode is an independent choice: exact, best-match, or threshold.*

expected answer is almost always empty, and a threshold array can report “nothing fired” without ranking anything. Table I summarizes the design space; notably, the cheapest cells support only Hamming distance, which is why encodings that hold accuracy under a Hamming metric are potentially valuable. A goal of what follows is to quantify what an emerging-technology CAM can or cannot buy: where the multi-bit cells of Table I change the achievable accuracy and energy, and where a plain CMOS Hamming array is already enough.

### B. CAM technologies

[DO THIS: MN: placeholder per your restructuring note – your technology-based CAM content goes here (SRAM vs. FeFET vs. RRAM implementations, the SUPREME/JUMP material, whatever you want to carry). The BEOL paragraph below is the seed; Table V in Sec. VII carries the device figures and could move up here if this subsection grows] The rungs of Table I differ in where the array can physically sit. SRAM-based binary and ternary cells are buildable in any CMOS process today, in the front-end silicon alongside logic. The FeFET-based cells trade that ubiquity for density and nonvolatility, and their device work is moving toward the back end of line (BEOL): monolithic-3D integration of high-endurance multi-bit FeFETs has been demonstrated with the memory tier above the logic tier [24], and BEOL-compatible ferroelectric transistors with AlScN films on two-dimensional channels have been reported at thermal budgets a metal stack tolerates [25]. BEOL RRAM and MRAM are already offered at 22/16 nm foundry nodes. A BEOL-capable cell is what would eventually let the signature bank sit in the metal layers above the compute that produces the activations; Sec. VII-C returns to what that placement would mean for this workload. Notably, none of said substrates would be built for one application: an SRAM-based associative processing unit already performs bit-serial Hamming search at billion-vector scale commercially [26], and CAM-based accelerators have been proposed across retrieval, one-shot learning, and range-matching workloads – so a monitoring part rides on a primitive several markets already use (Sec. II-C). [Sep 1, your (C): a brief echo of the associative-search-silicon paragraph, worked in here at the end of II-B; the full paragraph stays in II-C] [DO THIS: confirm the kim2023alsen author list be-

### C. Preliminaries and related work

**What a probe reads, and what it costs:** A token is carried through a transformer by a running vector, the *residual stream*, that every layer reads from and adds back to. After layer  $\ell$ , the result is  $h^{(\ell)}$ :  $d$  numbers ( $d = 8,192$  for a 70B model), one vector per layer, per token. The individual entries are not labeled features – no single coordinate means “deception” – so a detector has to read the vector as a whole rather than watch any one entry. A *linear probe* is the simplest detector over that object: one stored vector  $w$ , fitted offline on labeled activations, and a threshold  $\theta$ , with

$$w \cdot h = \sum_{i=1}^d w_i h_i > \theta \implies \text{flag.} \quad (1)$$

Probes of this form recover truth-related structure across several datasets and model families [9]–[11], and the directions are also actionable: adding a multiple of one back into the residual stream changes behavior, which is usually called steering [11]. Scoring one activation is  $2Ld$  multiply–accumulate operations for a model of depth  $L$  and residual width  $d$  – about 1.3 million operations for a 70B model, or roughly  $10^{-5}$  of the forward pass. This cost is analogous to  $L \cdot d$  rather than parameter count, and  $L \cdot d$  has not changed significantly even with frontier models (e.g., a 671B-parameter mixture-of-experts model has  $d = 7,168$  and  $L = 61$  [27] – an  $Ld$  smaller than a dense 70B model’s  $8,192 \times 80$  – so a bigger model could have a smaller probe vector).[two of your blue phrases were hard to read: I typed “is analogous to” and “changed significantly even with frontier models”; if the second was “now with,” say so]

**How detection is graded:** Quality is typically reported as area under the receiver operating characteristic curve (AU-ROC), i.e., the fraction of (deceptive, honest) pairs the detector orders correctly; 1.0 is perfect and 0.5 is random. An important “stress test” for library size is *cross-domain* – i.e., fit on one kind of dishonesty and evaluate on another [12].<sup>3</sup>

**Activation monitoring in production:** The two-stage cascade in [2], [3] and the deployed Gemini probes [4] are the closest published statements of the workload this paper aims to accelerate; both operate at  $K$  of order one. Monitoring has gotten cheaper as providers moved from reading text to reading activations: the first-generation text-classifier pair reported 23.7% overhead, while the probe-fronted cascade that replaced it reports 1–3.5% [2], [28]. Separately reported monitor overheads of roughly 0.1% of serving cost [29] are consistent with watching just one or two things. What the published figures do not tell us is how the 1–3.5% splits between the probe and the escalation stages; the  $2Ld$  arithmetic above bounds the probe’s share from below, at roughly  $10^{-5}$

<sup>3</sup>A *domain* is a subject matter to be dishonest about (what a word means, a checkable fact, a deduction); a *behavior* is a kind of dishonesty (lying about a fact, flattering the user, quietly underperforming on an evaluation).

of forward-pass compute. [DO THIS: still to verify: that the 1–3.5% in ccpp2026 is measured over the full cascade rather than the probe alone]

[Sep 2: the “Interpretability work that sets  $K$ ” paragraph is deleted per your full blue strike; all of its citations still appear in Sec. I and the red-team appendix]

**Approximate nearest-neighbor (ANN) search:** Locality-sensitive hashing [30], [31] and graph indexes [32] visit a small fraction of a bank and still retrieve most true neighbors at sub-millisecond query times [33]. Said methods are likely undesirable here, as a monitor cannot afford to miss – an adversary can aim at the approximation error, while an exhaustive scan has none to exploit (red-team appendix, App. A of the companion document).[Sep 1: trimmed per your note B – ANN acknowledged in two sentences, with the cannot-afford-to-miss reason; the full version and the queued adversarial-recall measurement stay in red-team objection (2)]

**Associative-search silicon:** Commercially, the nearest analog is an SRAM-based associative processing unit performing bit-serial Hamming search at billion-vector scale [26]. It is an existence proof for the primitive and the baseline a multi-bit or NVM-based part would have to differentiate against on density, energy, and metric. CAM-based accelerators have also been proposed for one-shot learning [20], nearest-neighbor search [21], and analog range matching [22]. As such, the hardware requirements of this workload are not “new”: the contribution here is the workload mapping and the library-size analysis rather than a new cell. [EXPAND: if we keep this paragraph, is there a group SUPREME/JUMP CAM application-survey citation you want here instead of listing three individual papers? The deck used the survey figure]

### III. A COST MODEL WITH ONE VARIABLE

[Aug 28 pm, per your second markup pass: Sec. III is now just the cost model, per your pick. Cut: the old III-A (redundant with the intro ladder and Fig. 1), the “dashed lines” paragraph, and Table III (your “seems redundant”). Folded into the intro: 122 runs across seven models. Kept here: the equations, crossings, anchors, the per-token/compounding argument, and the session-length point]

Let  $K$  be the number of stored signatures per monitored layer,  $d$  the residual width,  $L_{\text{mon}}$  the number of monitored layers,  $R$  the token rate, and  $b$  the bytes per stored element. On a GPU the bank is read once per monitored token, so

$$T_{\text{GPU}} = K d b L_{\text{mon}} R \text{ bytes/s,} \quad (2)$$

while a CAM broadcasts the query and leaves the bank in place, so

$$T_{\text{CAM}} = d b L_{\text{mon}} R \text{ bytes/s,} \quad (3)$$

independent of  $K$ . The right-hand axis of Fig. 1(a) reads Eq. (2) at  $d = 8,192$ , fp16, one monitored layer and  $R = 1,000$  tokens/s, against vendor HBM and LPDDR bandwidths. Monitoring traffic reaches an H100’s entire  $3.35 \text{ TB s}^{-1}$  [34] at  $K \approx 204\text{K}$  and an A100’s at  $K \approx 122\text{K}$ ; on a Jetson-class part at  $273 \text{ GB s}^{-1}$  the same crossing arrives at  $K \approx 17\text{K}$ ,

roughly an order of magnitude earlier. Those crossings move with the token rate, which is a deployment parameter rather than a property of the monitor, so Fig. 1(b) plots the limiting  $K$  across token rates spanning batch-1 edge decoding and a loaded datacenter node.

The anchor strip under Fig. 1(b) places familiar workloads on that axis. At the per-stream end, a single voice or chat user decodes at tens of tokens/s, and a deep-reasoning or agentic stream at a similar instantaneous rate – what distinguishes those tasks is duration, at  $10^3$ – $10^5$  generated tokens per query and up to  $10^8$  tokens per agent run [5]. Per-stream rates batch up: published serving benchmarks for 70B-class models report aggregate throughputs of order  $10^3$  tokens/s per datacenter GPU at high batch [35]. At the edge the two coincide, since a robot-control (vision–language–action, VLA) policy runs at batch 1 and its stream rate *is* the part’s aggregate rate: an autoregressive VLA at 3–6 Hz and  $\sim 7$  action tokens per step is tens of tokens/s, while a chunked  $\pi_0$ -class policy at the 19 Hz reported on a Jetson Thor emits 50-action chunks, i.e., of order  $10^3$  action tokens/s [36], and continuous video understanding pushes a backbone toward  $10^3$ – $10^4$  token positions/s. As such, the  $10$ – $10^4$  tokens/s span of Fig. 1(b) covers deployed practice rather than a hypothetical, and the workloads that reach the earliest crossings – batch-1 VLA and video on edge parts – also have no batch to amortize against. **[DO THIS: anchor numbers to verify before submission: the VLA rows are converted from VLA-Perf’s Hz figures (solid); voice, agentic-session, and video-backbone rates are order-of-magnitude from usage norms and frame-rate arithmetic, not from a single citable benchmark. The serving-node figure still cites a gray-literature roundup – swap in the interconnect number if you have one]**

Two properties of Eq. (2) justify the work in this paper. (1) Latency is the wrong way to judge a monitor’s cost. At small  $K$  the monitor’s reads hide under inference latency, so it *looks* free – but the bytes and the energy still must move. Rather, it is better to consider traffic and energy, and both are charged *per token*. (2) A per-token price multiplies a token base that is compounding: inference volume has been growing several-fold per year, so a fixed percentage of it is not a fixed cost. Long-running agents reinforce this point.[your “maybe” replacement applied; veto and it reverts to “make the same point concrete”] The campaign runs of Sec. I walk 32 steps over roughly 20 hours at run budgets up to  $10^8$  tokens [5]; a monitor for such a run is always-on by construction, and its per-token price is what decides whether it stays on.[Sep 1: footnote 4 (“no detector of ours has been run on cyber behavior”) deleted per your note, and the compact-shared-basis pointer sentence is cut – that evidence and the responses remain collected in red-team objection (1)]

[Aug 31: the contour figure (old Fig. 2) is cut per your X through it; fig\_contour\_v1 stays in the figures folder if it is ever wanted back]

[v03 note, still true: the full-width traffic-vs- $K$  figure (fig\_kscaling\_v1) stays retired; its content is Fig. 1]

## IV. MAPPING THE DETECTOR INTO A CAM

This section covers ways to map activation signatures onto CAM arrays. Candidate arrays span emerging devices through nearer-term CMOS solutions that leverage different encoding schemes – notably, traditional CAM designs can also be realized with nonvolatile memories (RRAM, MRAM, etc.), which could approach the density gains and, with a mostly static signature bank, benefit from nonvolatility. Sec. IV-A first spells out what any CAM-resident realization requires. The first mapping (Sec. IV-B) assumed the best-case device – a multi-bit, FeFET-class cell whose matchline computes the Euclidean distance the full-precision detector actually uses – and it is retained as the reference point the other encodings are measured against. The encodings of Secs. IV-C–IV-F satisfy the requirements on hardware buildable today, including Hamming-distance options that are near-term and realizable by a host of NVMs as well as by plain CMOS. [Sep 2, your (B): opener rewritten generically (“ways to map activation signatures to CAM arrays”), your right-margin NVM wording worked in, reference-point sentiment kept. The old “two constraints” sentence (deployment target + audit) is cut here since IV-A’s closing paragraph already carries the audit point – say the word if you want it restored]

### A. What “CAM-resident” requires

For the monitor to be one memory structure rather than a memory plus an accelerator, three properties have to hold. (1) The query has to be producible from the raw activation by something cheap, since whatever sits in front of the array runs on every token. (2) The comparison the array performs natively has to be the comparison the detector needs, which for a binary or ternary cell means the decision has to be expressible as a Hamming distance. (3) The decision rule has to consume what a matchline can report – a winner, a set of rows under a threshold, or a count – rather than a full sorted score vector.<sup>4</sup>

Property (1) also has a methodological consequence. If the front end is trained, it can learn to do the classification itself, leaving the memory as decoration. Sec. V-E describes the test we use to detect that, and what it caught when we ran it on pipelines that did have trained front ends.

### B. The first mapping: learned projection, 3-bit Euclidean

A multi-bit, FeFET-class cell broadens what a stored row can be: each cell holds a level rather than a bit, which could allow for a richer representation of the data – or the same representation in fewer memory cells – with the matchline computing the Euclidean distance the detector actually uses (Sec. II-A). A down-projection mechanism does need to be implemented regardless, since whatever sits in front of the array runs on every token (requirement (1)): each raw activation is reduced from  $d$  dimensions to a much smaller  $d'$  and quantized to a few bits per dimension.[Sep 2: opening reframed per your margin note – leads with what FeFETCAMs

<sup>4</sup>Returning all  $K$  scores is a readout job a CAM is not for.

can do (richer representation or fewer cells), with the down-projection-regardless point kept] Concretely, the pipeline took the residual-stream activation at layer 33 (the tuned layer choice described in Sec. V-A), built a bank of mutually distinct deception directions by iterative nullspace projection [37] (train a probe, project its direction out, retrain), weighted each stored direction with one trained scalar, compressed the activation with a *learned* down-projection from  $d = 8,192$  to  $d' = 128$  dimensions, and quantized query and stored rows alike to 3 bits with learned-step quantization [38], so that the low-precision rounding sits inside the training loop rather than being applied afterwards.<sup>5</sup> Matching was Euclidean winner-take-all, returning the winning row and its margin.  $d' = 128$  was chosen to match the row geometry of a multi-bit FeFET array – 128 cells sharing a matchline, at 3 bits per cell – so the stored row is a 384-bit-equivalent row (physically, 128 cells), which is where the matched 384-bit operating point used throughout Sec. V comes from.[Sep 1, three of your notes on this passage: (1) it now gives the strategy first ( $d \rightarrow d'$ , then quantize) and the numbers after; (2) “384-bit row” is now “384-bit-equivalent row,” per your “there are just 128 cells”; (3) the why-128 sentence ties  $d'$  to FeFET row geometry per your margin note – the 128-cells-per-matchline reading is mine, so fix the parenthetical if the real constraint is different. Layer 33’s proper introduction moved to the tutorial setup of Sec. V-A; this passage now just points there]

This mapping’s results – and the audit finding that retired its trained cousins – are evaluated in Sec. V (Fig. 2(b) and Table III).[Sep 1: the rest of this paragraph (“...the encodings in the rest of this section descend from the variants whose stored rows carry the classification”) is cut per your “can probably cut/scale this”]

The encodings that follow trade this mapping’s multi-bit cell for Hamming-distance solutions, which are near-term and realizable by a host of NVMs as well as by plain CMOS; a frozen projection matched on a multi-bit Euclidean matchline remains the natural recombination to try as the device matures.[Sep 2: the old Sec. IV-C (“The path back to a multi-bit cell”) is cut to this transition sentence per your notes – the “abandonment” framing you flagged as making no sense is gone with it, and the Hamming/NVM near-term point you asked for is in. The full old text is preserved as a comment in the source; the recombination-experiment question stays in the review appendix, and Sec. V’s audit subsection still points here]

#### C. Encoding 1: sign random projection, ternary (or via other NVMs)

A first option for meeting requirements (1)–(3) on hardware buildable today is a fixed random projection: draw  $P \in \mathbb{R}^{m \times d}$  once and freeze it, then encode an activation as the sign pattern of its projection,

$$c_j(h) = \text{sign}((Ph)_j), \quad j = 1 \dots m, \quad (4)$$

<sup>5</sup>Quantization-aware training at this precision is standard practice: a shipping on-device foundation model is trained with 2-bit QAT [39].

with a dead zone  $|(Ph)_j| < \epsilon$  mapped to don’t-care, which makes the code ternary and requires a wildcard-capable cell. This is the standard sign-random-projection (SimHash) construction [31]; the Hamming distance between two codes estimates the angle between the underlying vectors.[Sep 2, per your marks: opener simplified (“too AI-like”), the hyperplane explanation shortened per “CAM simplify,” and the “contribution here is not the construction” sentence deleted per your strike]

From a hardware perspective, the comparison is an XOR plus a population count, which a plain CAM array computes natively. A code wider than a practical array row is split across subarrays and the results aggregated, which is standard practice and a real design parameter – the measured FeFET-CAM demonstration of [23] uses exactly this organization, with voting to aggregate results between subarrays.

#### D. Encoding 2: thermometer coding (plain binary CMOS)

A second option for reaching a CAM realization – and one a plain CMOS binary array can serve: quantize each projected dimension into  $\ell$  levels and encode each level in unary, so a value of level  $v$  becomes  $v$  ones followed by  $\ell - v$  zeros. For two codes  $u, v$  over the same dimension,

$$d_{\text{Ham}}(\text{therm}(u), \text{therm}(v)) = \sum_i |u_i - v_i|, \quad (5)$$

i.e., Hamming distance on the code is exactly the  $L_1$  distance on the quantized values. The identity is exact, not approximate, which is what makes the encoding useful here: a cell that can only report disagreement now supports a magnitude-sensitive metric. No wildcard is needed, so the cell is a plain binary CAM at roughly 6–10 transistors rather than a TCAM at roughly 16.

The cell menu is wider than CMOS. A 1FeFET-1C array – a DRAM-like cell with the FeFET as access transistor – has been extended to ternary CAM operation, with a fabricated proof of concept whose sensed matchline voltage after charge sharing is linear in Hamming distance [40]. A plausible extension is to host either of this section’s encodings on such arrays: the Hamming linearity reads off the distances of Eqs. (4) and (5) directly, the don’t-care support serves the ternary code’s dead zone, and the DRAM-like cell promises density – especially considering rising 3D DRAM – with nonvolatile storage suited to a mostly static signature bank.[Sep 2, your two-arrow note at the identity claim: the 1FeFET-1C material is in, from the TED draft you attached (cited as “manuscript in preparation” – fix the entry when the author list settles). The DAC-thermosensitive.pdf citation and its extension still need the file from work] [DO THIS: DAC-thermosensitive.pdf: cite here with its plausible extension once the PDF arrives from MN’s work machine; also settle the fefet1c2026 bib entry’s authors before submission]

The comparison against sign-RP is therefore not symmetric. At a matched code length the two encodings use the same number of *bits* but not the same amount of *silicon*, and Sec. V reports both alongside that qualification.

### E. The decision rule

**[DO THIS: flagged per your margin note: have Indranil walk us through what the runs actually store and compare – the text below claims a few thousand labeled exemplars per domain with a  $k=31$  majority vote; if that is not what the runs do, this subsection, the audit discussion, and Table II’s bank sizes all change. (The round-2 log transcribed your note as “clarify with Ismail” – MN does not know an Ismail, so that read was wrong; item 7 of Indranil’s hand-off note carries this question.) No prose changed pending the answer]** The stored rows are labeled example activations rather than fitted directions – i.e., each domain stores not one direction per class but a few thousand encoded activations, each labeled honest or deceptive. A query is classified by the majority label among its  $k$  nearest stored rows under the array’s metric, with  $k = 31$  in the runs reported here (odd, so the vote cannot tie; the gain flattens well before 31). The scheme is analogous to few-shot classification: rather than one fitted direction per behavior (the  $N$ -way, 1-shot analog), the bank operates  $N$ -way, many-shot, with thousands of stored exemplars per domain and the array’s distance metric supplying the comparison. A vote over neighbors rather than a single winner is worth 3–5 accuracy points in in-distribution tests, and it consumes only what a best-match or threshold matchline can report.

### F. Encoding 3, as a ceiling: shortlist plus exact rerank

To bound what the compression costs rather than only measuring it, we also run a two-stage variant: stage 1 is a thermometer-Hamming search of the whole bank that keeps the top  $k$  rows as a shortlist, and stage 2 is an exact cosine rerank on the raw, uncompressed activations restricted to that shortlist. With  $k$  equal to the whole bank, stage 2 alone defines a full-precision ceiling for the same data and protocol. Shortlist-then-rerank is standard practice in large-scale retrieval (e.g., the two-stage GPU indexes of [33]); equivalently, stage 2 can be read as checking a smaller  $K$  that a GPU can easily and accurately serve, with stage 1 reducing the library to that size. The variant is also the architecture a real system might adopt if single-stage fidelity is inadequate, at the cost of a second memory structure and a floating-point stage.

## V. WORKLOAD 1: DECEPTION DETECTION

We first evaluate the mappings of Sec. IV on a deception detection workload, where the library is set by how many behaviors and domains someone decides to watch [Sep 2: your inserts (“a,” “workload”) and the cut of “on the first of the paper’s two workloads:” applied. Applying your strikes on “This section”/“evaluates” literally leaves no subject or verb, so the opener is “We first evaluate,” taking the “We 1st” you wrote above the heading as the cue – rewrite if you meant something else] – here, the  $K \approx 10^2$ – $10^3$  regime of Fig. 1. We describe the experimental setup, then report (1) the ten-domain fidelity of each encoding, (2) whether wider codes improve cross-domain transfer, (3) how the two-stage shortlist

behaves across bank sizes, and (4) the destroyed-vector audit that disciplines every configuration reported. Sec. VI evaluates the other end of the  $K$  axis, where the library is set by the vocabulary. [Sep 2: the “One may reasonably ask why we evaluate at small  $K$ ” paragraph (added last round) is deleted per your full strike. Your pink box next to it – whether Truthfulness connects to VLA, or to fall back on the established-dataset argument, or drop the question – is parked for a conversation rather than answered in the text; happy to brainstorm it in chat whenever]

### A. Experimental setup

Our evaluation case follows the cross-domain protocol of the Truthfulness Spectrum study [12]: a model produces honest and deceptive statements across several subject-matter domains, its activations are recorded, and a detector built from one domain’s examples is scored on every other domain’s – the “stress test” of Sec. II-C. Activations are taken from Llama-3.3-70B-Instruct [41]. The evaluation grid has ten domains: definitional, evidential, fictional, logical, ethics, sycophancy, `repe_honesty`, roleplay, insider trading, and sandbagging. The first four plus sycophancy correspond to the published grid [12]; `repe_honesty` follows the representation-engineering dataset [11]. Within each domain, 70% of examples form the stored bank and 30% are held out for test. Every cell of the grid is a cross-domain measurement: rows index the domain whose examples are in the bank, columns the domain the test activations come from. Bank sizes differ substantially by domain, from 831 rows (definitional) to 3,201 (sycophancy), which turns out to matter in Sec. V-D. Activations are read at layer 33: the source study selected that layer (with logistic-regression probes on average-token activations) from a cross-domain tuning sweep on its truth-type datasets, and we inherit the choice so our grids can be compared against theirs – i.e., it is a tuned choice for this model family and scale rather than a transferable constant. All numbers reported below are from cluster runs on this configuration. [Sep 1, per your “more tutorial in nature” note (and the page-4 “have we even mentioned layer 33” note, resolved as one fix): the subsection now opens by introducing the evaluation case, and layer 33 arrives later in the paragraph with the why – inherited from the source study’s tuning, per their Sec. 3. Sec. IV-B now points here instead of introducing the layer itself] **[DO THIS: confirm with I. Bera before submission: (1) that “evidential” is the same dataset the published grid calls “empirical,” and likewise ethics/ethical, roleplay/roleplaying and insider\_trd/insider\_trading (the published five truth types are definitional/empirical/fictional/logical/ethical); (2) whether the ternary dead zone was actually enabled in the 384-bit runs, or whether the code was binary in practice and “ternary” is a label carried over from the table headers]**

### B. The grids

[Sep 1: the “measurements here are representative of the  $K \approx 10^2$ – $10^3$  rungs” recap that opened this subsection is

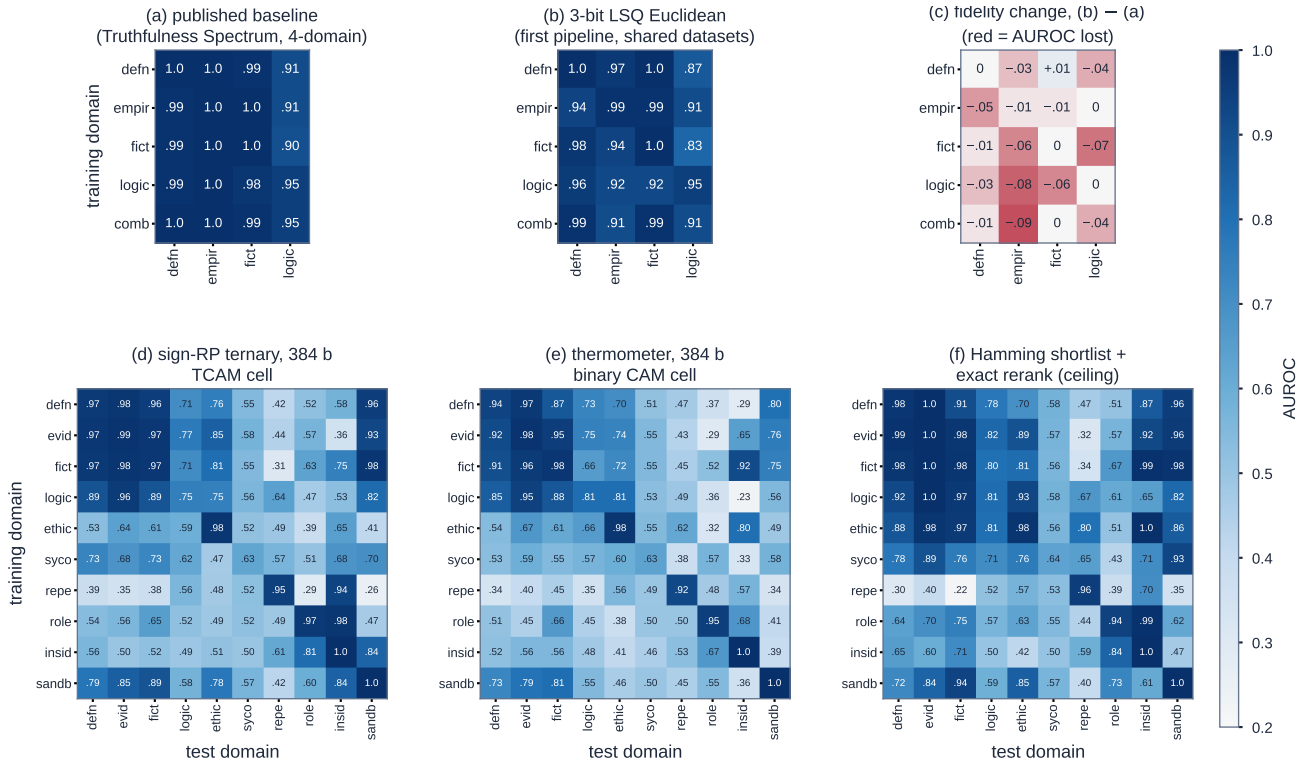


Fig. 2: Cross-domain AUROC, Llama-3.3-70B-Instruct activations at layer 33. Rows index the domain whose examples are stored in the bank, columns the domain the held-out test activations come from. Top row, four-domain published protocol: (a) the full-precision Truthfulness Spectrum baseline [12] as reproduced in our evaluation; (b) the first-pipeline Euclidean configuration – 128-d learned projection, 3-bit LSQ rows, Euclidean matchline (Sec. IV-B) – scored only on datasets shared with (a); (c) the cell-wise change, (b) minus (a). Bottom row, ten-domain protocol: (d) 384-bit sign-RP ternary code, requiring a wildcard-capable TCAM cell; (e) 384-bit thermometer code, requiring only a plain binary CAM cell; (f) Hamming shortlist over the whole bank followed by exact cosine rerank on raw activations, i.e., a full-precision ceiling for the same data and protocol. The two protocols use different datasets and splits and are not comparable cell by cell.

cut per your “Redundant? Can probably cut” – the reworked section intro above carries it]

Fig. 2(d)–(f) shows the ten-domain cross-domain grids for the two single-stage encodings and for the exact-rerank ceiling; Table II summarizes them alongside the hardware each requires. Each grid is read the same way: a row names the domain whose examples are stored in the bank, a column the domain the held-out test activations come from – i.e., a cell scores a bank built on behavior/domain  $X$  against test material from  $Y$ . [Sep 2, your (E): the how-to-read sentence, kept to one sentence since the cross-domain observation below already explains the off-diagonal reading (your note that (E) is partly covered there)] The top row of the same figure places the published four-domain baseline and the first-mapping Euclidean grid (Sec. IV-B) above them for reference; the two protocols differ in datasets and splits, so the rows of the figure are not comparable cell by cell. [Sep 1, your “why is our Fig. 2(d)–(f) richer than Fig. 2 in Truthfulness.pdf”: expected, not a data slip. Their Fig. 2 trains probes on only seven of their datasets (five truth types + sycophancy + expectation-inverted) and

tests on ten – roleplaying, insider trading, and sandbagging are test-only columns for them, since those on-policy benchmarks have no probe-training split. Our banks are stored labeled exemplars, so a bank can be built from *any* labeled dataset, and our grid legitimately has bank rows their lacks. Still queued for I. Bera: confirm the combined data feeding (d)–(f) is the current run, and the evidential/ethical naming flag in Sec. V-A][Sep 2, your “do we need to repeat/discuss both rows – let’s game this out”: no figure change made this pass. The tradeoff: (a) repeats Ying et al. and (c) is derivable from (a)–(b), so both are cuttable; (b) is our own Euclidean pipeline run on their protocol, with no analog in their paper. A middle path is a top row of just (b). Parked for your call]

The top row also scores the first mapping on its own terms. Against the published four-domain baseline, in-distribution AUROC was 1.00/0.99/1.00/0.95 versus the paper’s 1.00/1.00/1.00/0.95, the average cell-wise gap over the full grid was  $-0.03$ , and the single worst cell lost 0.09 (combined training, tested on empirical) – panels (a)–(c) of

Fig. 2.<sup>6</sup>[moved here from old Sec. IV-A per your restructure. Standing question from Aug 28: this text matches the uGrads’ deck (mean gap  $-0.03$ , worst  $-0.09$ ); the Aug 14 methods document said  $-0.06$  / within  $0.05$  – worth asking which run is current]

We make three observations on the ten-domain grids.

**In-domain fidelity is essentially unaffected by the encoding:** When a bank is tested on the domain it was built from, the compressed codes give up almost nothing: the diagonal means are 0.920 (sign-RP ternary, 384 b) and 0.919 (thermometer, 384 b), against 0.928 for the exact rerank. Said differently, replacing full-precision cosine similarity over the raw 8,192 values with a 384-bit code in a plain array costs 0.008–0.009 AUROC. The factual block (definitional, evidential, fictional – all nine cells, transfer included) averages 0.974 for ternary and 0.942 for thermometer, against the 0.978 ceiling.[Sep 2: rewritten in plain English per your note, voice rules applied; every number unchanged]

**Cross-domain fidelity is where compression shows, and it separates the two encodings:** An off-diagonal cell scores a bank built from one domain against test examples from another, so it measures how well a stored library transfers to material it was not built from. That transfer is where the encodings pay: off-diagonal means are 0.640 for ternary and 0.579 for thermometer, against 0.705 for the full-precision ceiling – a cost of 0.065–0.126 AUROC, where in-domain the same compression cost less than 0.01. The ordering is the surprise. At this matched 384-bit point, ternary is ahead of thermometer by 0.061 AUROC on average and in 63 of 90 cells, which is the opposite of what a preference for the cheaper cell would predict. One caveat keeps the question open: the comparison is matched in *bits* but not in *silicon*, since a ternary cell needs wildcard capability. If a ternary position really costs two bits of hardware, a matched-silicon comparison would give thermometer roughly twice the code length; that experiment has not been run. **[DO THIS: this is the paragraph to argue about. Options: (1) report both and leave it open, as drafted; (2) run the matched-cell-count experiment first and lead with whichever wins; (3) lead with thermometer on the hardware-simplicity argument and footnote the ternary advantage. I would pick (2) if there is student time before the deadline, since a reviewer will ask exactly this]**

**The pattern of failure reproduces the published one:** All three grids keep the factual block high and put sycophancy near chance from every other row, which is the qualitative result the source study reports [12]. Sycophancy’s own in-domain cell is the weakest diagonal entry in every configuration (0.627, 0.633, 0.636), stable across conditions, so it appears to be a property of the data rather than a compression artifact. The weakest off-diagonal cell sits at a similar depth in every configuration (0.259, 0.226, 0.220) but its *location* moves with the encoding: `repe_honesty`  $\rightarrow$  `sandbagging`

<sup>6</sup>Those values are in-distribution AUROC on a four-domain grid and a different protocol from the ten-domain grid below; Fig. 2 keeps the two protocols in separate rows, and they are not comparable cell by cell.

TABLE II: The encodings, their measured fidelity on the ten-domain grid, and what each needs from a cell. Bank size is for a 4,000-row exemplar bank at the given code width. The in-domain minimum is sycophancy in every row (0.627 / 0.633 / 0.640 / 0.636 top to bottom); the cross-domain minimum is 0.259 / 0.226 / 0.219 / 0.220 top to bottom, and its cell location varies by encoding (Sec. V-B).

| Encoding               | in-dom.<br>mean | cross-dom.<br>mean | Cell needed                | Bank<br>(4K rows) |
|------------------------|-----------------|--------------------|----------------------------|-------------------|
| sign-RP ternary, 384 b | 0.920           | 0.640              | TCAM, $\sim 16T$           | 192 kB            |
| Thermometer, 384 b     | 0.919           | 0.579              | BCAM, $\sim 6\text{--}10T$ | 192 kB            |
| Shortlist + rerank     | 0.928           | 0.671              | BCAM + FP unit             | 192 kB + raw      |
| Exact rerank (ceiling) | 0.928           | 0.705              | FP baseline                | 65 MB             |

for ternary, logical  $\rightarrow$  insider trading for thermometer, and `repe_honesty`  $\rightarrow$  fictional for the rerank configurations. The bigger picture is that the accuracy ceiling on transfer is set by the representations, not by our compression – the full-precision grid has the same holes, in the same places. Covering a behavior suite at this accuracy therefore means storing directions for each behavior and domain rather than hoping one direction travels, i.e., operating at the library scale this paper prices.[Aug 28: sanity check against the plotted arrays, per your student’s flag. The earlier text claimed `repe` $\rightarrow$ fictional was the worst cell in every configuration; the values 0.259/0.226/0.220 are each configuration’s minimum, but the cell differs for ternary and thermometer. Every other quoted statistic in this section verified against the arrays to the stated precision. CLOSED Aug 28 night against his source files: all four plotted grids match the methods document cell-for-cell (zero differences in all 400 cells), and the transposed comparison fails in all 90 off-diagonal cells per grid, so there is no train/test transposition – the figure plots his data exactly. His document’s own min/max/mean lines also agree with every aggregate quoted here, and it names `repe` $\rightarrow$ fictional as the min driver only for the two rerank grids, consistent with the corrected text above]

[Aug 28: the top row is the “all of this in 1 figure” request. Values transcribed cell-by-cell from slide 4 of the uGrads’ deck (Gisiner/Haidinger/Whalen); every cell was present, so no white placeholder squares were needed. The  $\Delta$  panel recomputed from (a) and (b) matches the deck’s printed  $\Delta$  in all 20 cells, a consistency check on the transcription. Panel (b) is the same 3-bit LSQ Euclidean pipeline Sec. IV-B describes – its diagonal 1.00/0.99/1.00/0.95 matches the values quoted there. Cut (c) if the figure needs to shrink; (a) and (b) carry the claim]

C. Cross-domain transfer is hard, and wider codes do not close the gap

The transfer gap of Sec. V-B raises the natural question of whether more code capacity would close it – and the answer matters, because if it does not, covering more behaviors means storing *more* comparison vectors rather than wider ones, i.e., growing  $K$ . Widening the code should tighten the angle

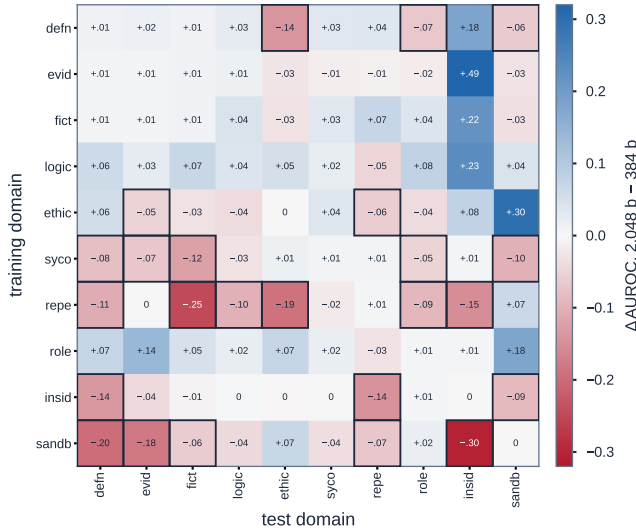


Fig. 3: The cost of a wider code, cell by cell: the change in cross-domain AUROC as the sign-RP ternary code grows from 384 to 2,048 bits (red is fidelity lost; off-diagonal cells losing more than 0.05 are outlined). The diagonal is flat to slightly positive while the transfer losses concentrate in the rows already weakest at 384 bits, so the extra capacity sharpens domain-specific boundaries rather than helping them travel (Sec. V-C).

estimate in Eq. (4) and improve matching, and in-domain it does. Cross-domain it does not, in either encoding.[Aug 31: reframed per your bottom-margin note (“cross-domain transfer is hard so we need more comparison vectors”). No push-back on the substance – it is the same argument the red-team appendix already makes – but I kept the claim conditional on this one model and grid, as objection (1) requires] Growing the code from 384 to roughly 2,048 bits, 23 of 90 off-diagonal cells drop by more than 0.05 AUROC for the ternary code, with a worst single-cell change of  $-0.303$  (sandbagging tested on insider trading); for the thermometer code the same growth moves 12 of 90 cells by more than 0.05, worst  $-0.209$ . Fig. 3 maps the change cell by cell for the ternary code: the diagonal is flat or slightly up, the losses concentrate in the rows that transfer worst at 384 bits (repe\_honesty, sandbagging, sycophancy), and the one column that improves broadly – insider trading – is a domain, not the encoding, getting easier to hit.

A plausible reading is that extra dimensions preserve more of the geometry of the training domain, including the parts that do not generalize, so a wider code is a more faithful encoding of a domain-specific decision boundary. Whatever the mechanism, the takeaway for hardware is simple: (1) the cheaper array is also the one that transfers better – on this grid, the costlier cell does not buy accuracy – so there is no argument for paying more silicon per row; and (2) if transfer must come from somewhere, it comes from storing

more vectors, not wider ones – which is the library growth Sec. III prices.

[Aug 28 night, from the student’s files: the 2,048-bit ternary grid exists (slide 2 of his older deck; saved as ternary2048.npy), so this  $\Delta$  figure per your pick; the worst cell is sandbagging→insider at  $-0.303$ , now named in the text. The 2,048-bit thermometer grid is still aggregate-only (12/90,  $-0.209$ ). One count note: my recount from his 3-decimal grids gives 24 cells past  $-0.05$ , not 23 – two cells sit at exactly  $-0.051$  (ethics→evidential, syco→roleplay) – so his 23 presumably comes from unrounded data; text keeps his count, and the figure outlines 24]

#### D. A fixed shortlist does not travel across bank sizes

The two-stage variant of Sec. IV-F is the natural way to recover the ceiling, and at first reading it works: with an absolute shortlist of  $k = 400$ , the diagonal mean is unchanged at 0.928 and the off-diagonal mean falls only from 0.705 to 0.671. The aggregate hides a failure of a *fixed* parameter, which matters more for a hardware design than for a software one.

Bank sizes differ by roughly  $4\times$  across domains, from 831 rows (definitional) to 3,201 (sycophancy). Fig. 4(a) measures coverage: how often the true full-precision top-1 neighbor actually lands inside the Hamming shortlist. For the definitional bank the mean is 82.5%, against a chance floor of  $k/N = 48.1\%$ . For the sycophancy bank the mean is 18.3% against a floor of 12.5%, i.e., barely above chance, and several test domains sit at or below their own floor. Consequently, 8 of the 10 training rows are stable between  $k = 400$  and full precision while the sycophancy row loses 0.204 AUROC on average and up to 0.468 in a single cell; the ethics row shows a smaller version of the same effect at  $-0.079$ .

Fig. 4(b) shows this is a sizing problem rather than a broken mechanism. Sweeping  $k$  as a fraction of each domain’s own bank, sycophancy’s cross-domain mean recovers from 0.532 at 12.5% to 0.736 at 100%, mostly monotonically, and the 100% value matches the independently computed full-precision row average to three decimal places (0.736 vs. 0.736; definitional 0.750 vs. 0.751), which is a useful self-consistency check on the pipeline. The recovery costs 75–100% of the bank, at which point it is not a shortlist. For a CAM-resident design, that argues for the single-stage encodings above, or for a threshold rather than a top- $k$  criterion so the number of survivors adapts to the bank instead of being fixed at design time.

#### E. The destroyed-vector audit

Any pipeline of the form “trained front end, then memory” can pass its accuracy test while the memory contributes nothing. We therefore re-measure every configuration with the stored rows replaced by random vectors of the same shape: a CAM-based detector should collapse to chance when the bank is destroyed, and if it does not, whatever sits in front of the array was doing the classification. We report the accuracy that

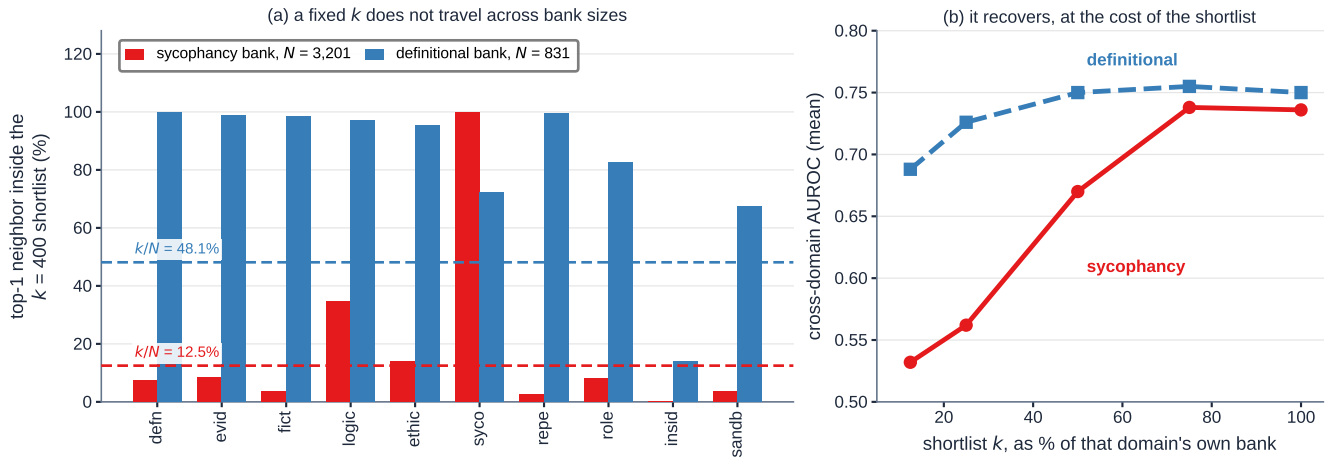


Fig. 4: Why a fixed shortlist size is a hardware problem. (a) Coverage: how often the true full-precision top-1 neighbor is inside the  $k = 400$  Hamming shortlist, for two banks that differ by  $\sim 4\times$  in size, with each bank’s chance floor  $k/N$  marked. (b) Cross-domain mean AUROC as  $k$  grows as a fraction of each domain’s own bank. Recovery is real but costs most of the bank.

TABLE III: The destroyed-vector audit across five variants of the first pipeline (July 30). Front-end share is the accuracy still present after every stored row is replaced by a random vector, i.e., the share the trained layers were carrying alone. High is bad: 100% means the memory contributed nothing. This audit is our sanity check that a pipeline is not “cheating,” i.e., classifying in a trained front end rather than in the stored bank; we run it on every configuration reported in this paper.

| Variant                    | Front-end share | Outcome  |
|----------------------------|-----------------|----------|
| Learned binary hash        | 98–100%         | retired  |
| Weighted thermometer       | 66–85%          | retired  |
| 3-bit LSQ, Euclidean       | 24–28%          | survives |
| Direct sign, no projection | 20%             | survives |
| Random sign projection     | 4–14%           | survives |

survives destruction as the *front-end share*, where high is bad: 100% means the memory was decoration.

The audit has changed decisions. Table III reports it across five variants of the first pipeline (Sec. IV-B). In the two highest-scoring variants, one of which reached 0.993 AUROC, between 66% and 100% of the accuracy survived randomizing every stored row: the trained layers in front of the array were performing most of the classification, and the CAM was close to decoration. Both were retired. The 3-bit LSQ Euclidean configuration sits at 24–28%, so its memory is contributing; the variants whose stored rows carry nearly everything are the ones with a frozen or absent projection, at 4–20%, which is why the encodings of Secs. IV-C and IV-D use frozen projections – with nothing trained in front of the array, the failure has nowhere to occur.

The encodings reported above pass. Randomizing every stored activation collapses the sign-RP grid to a mean of 0.50, with 83% of cells between 0.4 and 0.6, so the classification

is coming from what is stored rather than from anything in front of the array. An accuracy number alone would not have separated the five variants of Table III – the highest-scoring one was the worst offender. As such, we now run the audit by default on every configuration, report front-end share alongside AUROC, and treat a variant whose accuracy survives bank destruction as an accelerator result rather than a memory result. The recombined design of Sec. IV-B should pass this audit by construction – nothing is trained ahead of its array – and will be checked the same way when it is run.

[Aug 28 night: verified against the full audit grid in the student’s older deck (slide 3) – mean 0.504, 83.0% of cells in  $[0.4, 0.6]$ , so the quoted figures are exact. That grid is the 2,048-bit configuration, though, so the standing question is narrower but still open: confirm the audit was also run at 384 bits, or say “at 2,048 bits” in this sentence]

## VI. WORKLOAD 2: WHOLE-VOCABULARY READOUT

This section evaluates the mappings on the second workload: whole-vocabulary readout, where the library is set by the model’s vocabulary rather than by anyone’s choices – the  $K \approx 1.3 \times 10^5$  rung of Fig. 1. We describe the published method and why its  $K$  is structural, report an in-house reproduction at watchlist scale with its measured fidelity at CAM precision, and price what full readout would cost a CAM-resident design against streaming the same bank from HBM. [Aug 31: section intro added per your “frame this section like V” note]

### A. A workload whose $K$ nobody selected

The grids in Sec. V are representative of the  $K \approx 10^2$ – $10^3$  regime, where the library grows with the number of behaviors somebody thinks to watch. A recently published readout method sets  $K$  a different way. It reports that a model holds words “in mind” without emitting them, and that for

every token in the vocabulary there is an internal pattern meaning that word is currently active, readable whether or not the word is ever said [15]. In the spider example of Sec. I, the word appears in neither prompt nor answer, and replacing the internal *spider* pattern with the one for *ant* changes the answer from eight to six. The same work reports that the readout catches misbehavior beyond the staged safety test: “manipulation” was active while a model produced falsified data during a pre-release audit.

For our purposes the relevant property is structural rather than behavioral. Reading this space means comparing the internal state against one stored pattern *per vocabulary token* – roughly 128K signatures for a Llama-family tokenizer – at several layers, every token. Nobody chose that library size; it falls out of the vocabulary. And it is plausibly a floor, since the published method tracks single words, and phrases would only enlarge the library.

### B. An in-house reproduction, at watchlist scale

We reproduced the readout in-house on Llama-3.1-8B: a layer-by-layer ranked list of what the model is currently representing. About 13 of 32 layers carry a usable readout at this scale, which is the source of the “13–25 usable layers” factor used in Sec. III. Each signature is precomputed offline, so a watched word costs one comparison per token, and we stored 47 deception-adjacent signatures per monitored layer as CAM rows.

In a staged scenario in which the model is told it is about to be replaced and is given a personnel file it was not meant to use, ten watch words were tracked by exact full-vocabulary rank across the 2,048 prompt tokens the model reads before generating anything. Five of them – “scandal,” “evidence,” “leaked,” “fake” and “shutdown” – reached rank 0 at interpretable moments in the reading, “threat” reached rank 1 and “blackmail” rank 2, and none of the ten appears in the prompt text.[\[corrected Aug 21: the v01 draft said three words at rank 0, which is how the number appears in every downstream deck. The 07-16-26 meeting notes are the source of record and say five, plus threat at 1 and blackmail at 2\]](#) The reply the model then produced was cooperative and unremarkable, i.e., not something a text classifier would score as harmful. Ground truth for the reproduction is the model’s own full-precision readout, so no labeled deception data is required anywhere.

These are preliminary numbers: one model, one staged scenario, ten watch words in the demonstration and 47 stored signatures.<sup>7</sup>

### C. Fidelity at CAM precision

The deception grid of Sec. V needs labeled activations. Readout does not: the full-precision, full-dimension dot product against the stored patterns *is* the ground truth, so a compressed implementation can be scored directly against

<sup>7</sup>Several of the fidelity and scaling values in this subsection were read from result charts rather than from result files; verification against the team’s files is queued, and the scaling sweep re-runs with three known fixes before any of it is reused externally.

the exact readout it is trying to reproduce. Compression can therefore be measured here without labels, and the deception numbers of Sec. V are not assumed to carry over to it.

The setup takes J-space vectors at a fixed layer, gathers residual-stream activations from Wikitext, records which stored vectors fire above a global threshold under the exact computation, and asks how well each CAM-compatible encoding reproduces that list. Two readouts are scored: *threshold*, where every row that fires is reported and the metric is per-row firing F1, and *top-1*, where the single closest row is reported and the metric is agreement with the exact readout’s choice. The bank is 47 deception-adjacent vectors per layer, as in Sec. VI-B.

Fig. 5(a,b) plots both against storage bits per CAM row, which is the axis that connects fidelity to Sec. VII: bits per row set the bank size, and the bank size sets what has to be held or moved. An uncompressed fp16 row at  $d = 8,192$  is 131,072 bits and reproduces the exact readout by definition. Three observations.

**Two orders of magnitude of compression cost less than it might:** A 3-bit Euclidean code at  $\sim 10^3$  bits per row – a  $128\times$  reduction – reaches 0.96 top-1 agreement and 0.81 per-row firing F1. Keeping the projection in full precision and quantizing nothing but the width (the exact- $L_2$  series) reaches 1.00 top-1 at 2,048 bits and 0.84 firing F1 at 4,096, so most of the remaining gap at  $10^3$  bits is the quantizer rather than the projection.

**The top-1 readout is markedly cheaper than the threshold readout:** At every code width the top-1 curve sits above the firing-F1 curve, by 0.15–0.2 in the range that matters. Reporting *which row is closest* survives compression better than reporting *which rows cross a threshold*, because the first is a comparison among rows and the second is a comparison against an absolute value that quantization shifts. For a CAM this is a matchline-mode result: best-match mode tolerates a cheaper cell than threshold mode does, at equal fidelity.

**Hamming and thermometer codes do not reach the same operating point here:** Both rise with code width to roughly 0.74 top-1 and 0.49 firing F1 at  $10^3$  bits, and the Hamming series then falls sharply at 4,096 bits, where the encoding switches to a direct sign hash without a learned projection. That is the opposite ordering from Sec. V, where at a matched 384 bits the sign code was ahead. The two workloads are asking different questions – separating two labeled classes there, reproducing a specific ranked list here – and the metric a readout needs appears to be the one a multi-bit cell provides. Sec. VII-D takes up what this would mean for the array if it survives the checks queued in Sec. VII-F. [\[v03: per the v02 note, the design-space consequence of this disagreement now has its own paragraph in Sec. VII-D, and the projection-artifact check is item \(3\) of the measurement plan in Sec. VII-E\]](#)

### D. Fidelity as the library grows

Compression fidelity at a 47-row bank does not establish anything about a  $10^5$ -row one, so the same pipeline was swept

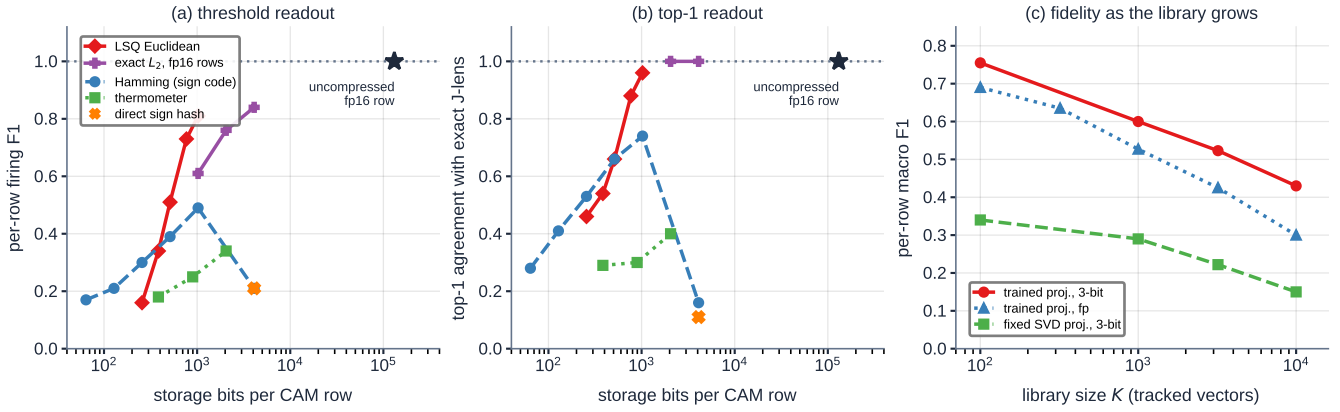


Fig. 5: Whole-vocabulary readout reproduced through a CAM. (a) Per-row firing F1 and (b) top-1 agreement with the exact J-lens, against storage bits per CAM row, for a bank of 47 vectors per layer; the star marks an uncompressed fp16 row (131,072 bits), which reproduces the exact readout by definition. (c) Per-row macro F1 as the library grows from  $10^2$  to  $10^4$  tracked vectors at 124+4 cells per row. Values digitized from the source deck; see the footnote in Sec. VI-C for the limitations that apply to panel (c).

over library size. Fig. 5(c) reports per-row macro F1 from  $K = 10^2$  to  $K = 10^4$ .<sup>8</sup>

Three things follow. **The library stays alive:** the fraction of rows that remain distinguishable is essentially 1.0 out to  $10^4$  tracked vectors for a trained projection, with a fixed SVD projection beginning to shed rows at about  $10^4$  (to 0.88). Nothing collapses. **Per-row fidelity nonetheless degrades:** macro F1 falls from 0.755 at  $K = 10^2$  to 0.43 at  $K = 10^4$  for the 3-bit trained projection. Against an always-fire baseline that drops from 0.10 to 0.005 over the same range, 0.43 is still nearly two orders of magnitude better than trivial, but it is not a fidelity anybody would deploy against. **The degradation is not a quantization artifact:** the full-precision projection degrades faster, from 0.69 to 0.30, and is overtaken by the 3-bit trained projection above  $K \approx 10^3$ . What gets harder as the library grows is the discrimination, not the arithmetic, which is a result in favor of the compressed implementation rather than against it.

In engineering terms, holding  $10^4$  rows is not the problem and reading them faithfully is. That places the open work on the encoding and the readout mode rather than on the array, and it is the reason Sec. VII prices capacity and traffic separately.

#### E. What full readout would cost

Priced at 70B geometry, an uncompressed bank of 128K signatures at  $d = 8,192$ , fp16, is 2.1 GB per layer. Streaming that on every token at 1,000 tokens/s over 25 monitored layers

<sup>8</sup>Panels (a) and (b) are digitized from the charts in the group’s 07-30-26 result deck; the underlying result files were not available. Panel (c) is from the same deck and carries three limitations its author states: an epsilon-inside-square-root bug drove the  $K = 32$  and  $K = 320$  points to zero and they are dropped here; mean-token pooling was used by mistake, so the residual-stream vectors differ slightly from the per-token vectors a deployment would see; and CAM top- $k$  clipping is assumed away, i.e., a row that fires is taken to be selected. A re-run with all three fixed is queued.

is roughly  $53 \text{ TB s}^{-1}$ , which is not a workload anyone runs. The useful part of the answer, however, stays small: only on the order of 25 patterns are meaningfully active at any one time, so the operation is detection rather than readout – scan everything, return the few that fired – which is the question a threshold-mode CAM answers natively.

Three things would make it tractable, in increasing order of what they require.

**A watchlist:** Store the few hundred words already known to be worth watching, which drops  $K$  by three orders of magnitude. Our 8B demonstration in Sec. VI-B is this case. A watchlist catches only what somebody thought to list, and the words that matter most may be the ones nobody anticipated, so it is a first rung toward full readout rather than a substitute for it.

**Threshold matching instead of ranking:** A CAM can report which rows are within  $\tau$  without scoring the rest, and with roughly 25 active patterns out of 128K nearly every query returns nothing.

**Compressing the stored side:** At the code widths Sec. V measured, 384–2,048 bits per row against 131,072 bits for an uncompressed fp16 row, the whole 128K bank is 6–33 MB per layer (Fig. 6), which fits on a die. The requirement is that it not move. Only the first of the three has been demonstrated here; the other two are design arguments.

## VII. HARDWARE IMPLICATIONS

### A. Monitoring as a share of the host’s own traffic

Eq. (2) is easier to interpret against what the model itself already moves. In the datacenter, a 70B model at fp8 reads roughly 70 GB of weights per token; a library of  $K = 100\text{K}$  rows at  $d = 8,192$  adds 1.6 GB, or about 2%. At the edge, an 8B model at 4-bit reads roughly 4 GB per token, and  $K = 25\text{K}$  at  $d = 4,096$  adds 205 MB, about 5%, rising to roughly 20% by  $K = 100\text{K}$ . The same architectural point therefore

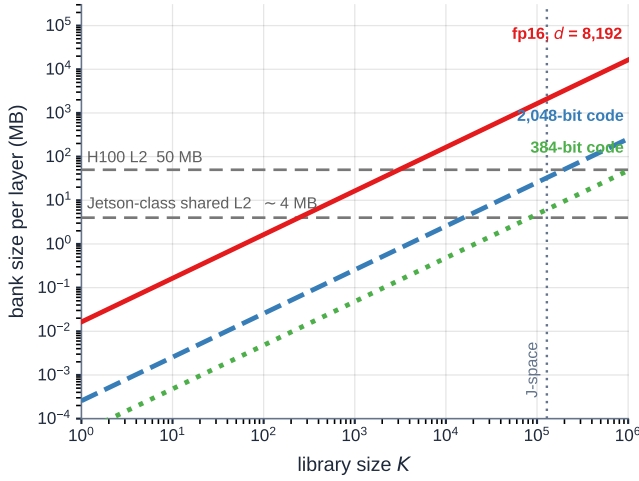


Fig. 6: Bank capacity per monitored layer against library size, for uncompressed fp16 rows and for the two code widths measured in Sec. V, with two on-chip capacity references. A whole-vocabulary bank at the measured code widths is 6–33 MB per layer; the same bank uncompressed is 2.1 GB.

arrives one to two orders of magnitude earlier in  $K$  at the edge, on parts that have no HBM available at any price. Edge inference is already bandwidth-limited, so every monitoring byte trades approximately 1:1 against token rate, and the 15–60 W envelope makes DRAM energy per bit a first-order term rather than a footnote.

#### B. The cache cliff, and how close current banks are to it

Capacity produces a sharper discontinuity than bandwidth. Consider a Jetson-class part: 273 GB s<sup>-1</sup> LPDDR5X, roughly 4 MB of shared L2 and no HBM [42], running an 8B assistant onboard and monitoring only the four behaviors observed in the campaign of Sec. I. At  $d = 4,096$ , fp16, 500 stored rows is 4.1 MB, which is that shared L2 almost exactly. Below the cliff the bank is resident and monitoring is close to free; above it the bank streams from LPDDR on every token.

The arithmetic of Sec. I put four behaviors at 520–1,000 rows, and the banks currently in hand are already in that range: 47 signatures across 13 layers is roughly 600 rows, and the exemplar bank of Sec. V is about 4,000. The fair pushback is that at the code widths Sec. V measured, today’s banks compress back under the cliff – 4,000 rows at 2,048 bits is about 1 MB. Two qualifications apply. Compression has been validated for detection, not for the steering-grade full-precision directions that a correcting monitor would need. And the cliff does not move as the library grows: at whole-vocabulary scale even the compressed bank is out of cache (Fig. 6). The edge crossing is therefore not purely a 10<sup>5</sup> hypothetical; the libraries already in hand sit near the threshold.

#### C. Where the search engine could sit

Table IV lists the placements and what each trades. The trade is always the same: how far the query has to travel,

TABLE IV: Placements for a CAM-resident monitor.

| Placement                            | Trade-off                                                                                                                    |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Co-resident on the inference GPU     | Contends with live inference for cache and bandwidth; the contention term is not in our numbers and could dominate at scale. |
| Dedicated monitoring GPU(s)          | Whole GPUs scanning a static table, plus cross-device latency.                                                               |
| CAM or CXL-attached appliance        | Matches in place, but the query crosses a board-level bus; a separate part to build and qualify.                             |
| BEOL search engine above GPU compute | Query travels microns, no cache contention, non-volatile static library; no PDK support for BEOL FeFET today.                |

and what it has to contend with on the way. A monitor co-resident on the inference GPU competes with live inference for cache and bandwidth, which is the contention the analysis above assumes away and which would need to be measured. A back-end-of-line (BEOL) search engine in the metal layers above GPU compute keeps the query travel to microns and the library nonvolatile and static, so it augments a part already purchased – but no foundry PDK offers BEOL FeFET today, and BEOL RRAM/MRAM at 22/16 nm is the defensible near-term citation (the BEOL device work is in Sec. II-A), which makes that row a 2030s option and it should be labeled that way. **[DO THIS: confirm you want the BEOL row in a DATE paper at all. It is the most speculative entry in the table and DATE reviewers are less forgiving than a sponsor audience. The CXL/appliance row is the one an industrial reviewer can price today]**

#### D. Device candidates

Table V lists candidate substrates. The encodings of Sec. IV matter here because they change which row is required. A thermometer code needs only the last row, i.e., no new device: a binary SRAM CAM in any CMOS process. The sign-RP ternary code needs wildcard capability, available in CMOS TCAM at higher area or in a 2-transistor FeFET cell [20]. A multi-bit Euclidean matchline is the row that requires a multi-level device [21]. The two-stage variant would benefit from it, as would the first pipeline of Sec. IV-B, whose 3-bit Euclidean matching maps directly onto such a cell, and the frozen-projection Euclidean design of Sec. IV-B. The multi-bit row therefore stays in the table even though the encodings this paper measures do not require it: it is the substrate for the next experiment rather than a device the current results depend on.

**The two workloads may want different cells:** The measured results give the multi-bit row a second reason to stay. At a matched 384 bits the sign-RP ternary code is ahead of thermometer on the deception grid (Sec. V), while on the readout workload the 3-bit Euclidean code is well ahead of every Hamming-metric encoding at the same storage budget (Sec. VI-C). The two workloads are asking different questions – separating two labeled classes there, reproducing a specific ranked list here – and the ordering flip is consistent with the readout needing the magnitude information that sign

TABLE V: Candidate substrates for the signature bank. Device figures are literature values, not our measurements.

| Tech     | Bits/cell | Write        | Endurance           | Best fit here          |
|----------|-----------|--------------|---------------------|------------------------|
| FeFET    | 2–3       | ~20 ns       | $>5 \times 10^{10}$ | multi-bit Euclidean    |
| ECRAM    | up to 6   | ~200 $\mu$ s | $\sim 10^{12}$      | static direction bank  |
| RRAM     | 1–3       | sub- $\mu$ s | $\sim 10^9$         | density; wear-leveling |
| SRAM CAM | 1         | SRAM         | –                   | Hamming; any CMOS fab  |

codes discard. If the flip survives the two checks queued in Sec. VII-F (matched cell count, and a common projection across encodings), it is a design-space result: a monitoring engine that has to serve both ends of Fig. 1 either provisions two cell types or pays for the multi-bit cell and serves the small- $K$  workload with it too. If it does not survive them, one array serves both. Write endurance appears in the table because the bank is rewritten whenever the monitored model is updated or the probe layer is re-tuned, which is a per-model-release event rather than a per-token one; that is the property that makes an NVM bank reasonable here and unreasonable for transformer intermediates.

#### E. A first-order energy and latency estimate

The analysis above is capacity and bandwidth arithmetic. A full cost model against a tuned GPU baseline is future work (Sec. VII-F); what can be stated now, from published device and memory figures, is a first-order estimate of the energy and latency of the search itself, and the gap it indicates is wide enough to survive considerable error in the inputs.

On the GPU side, the dominant energy term at large  $K$  is reading the bank out of DRAM. Measured HBM2 access energy is approximately 3.97 pJ/bit [43]. At the whole-vocabulary operating point –  $K = 1.3 \times 10^5$  rows of fp16 at  $d = 8,192$ , i.e., the 2.1 GB bank of Sec. VI-E – reading the bank once is roughly *67 mJ per token per layer*, before any arithmetic is charged. On the CAM side, the same search is  $K$  rows of 2,048-bit code compared in place, about  $2.6 \times 10^8$  cell comparisons. Published figures for binary and ternary CAM search sit in the range of 0.1–1 fJ per cell per search across CMOS survey data and fabricated FeFET arrays [19], [20], [44], which puts the whole-bank search at roughly *0.03–0.3  $\mu$ J per token per layer* – five to six orders of magnitude under the DRAM read. The gap decomposes into two factors:  $64\times$  from the code width (2,048 bits stored per row against 131,072; measured fidelity at that compression is what Secs. V and VI establish) and  $\sim 4 \times 10^3$  from the substrate (fJ-scale cell comparison against pJ-scale DRAM bit movement). As such, even a  $10\times$  error in the per-cell figure – which spans the published range – moves the conclusion by one order out of five or six. Scaled to a deployment, the same arithmetic at 25 monitored layers and  $10^3$  tokens/s is roughly 1.7 kW of DRAM access power on the GPU side against tens of milliwatts of array search: monitoring at this  $K$  is not a workload a deployed part absorbs, which is what Fig. 1(b)

TABLE VI: The cost model: measured rows, first-order estimates, and open entries. EST marks first-order values from literature figures (HBM at 3.97 pJ/bit [43]; CAM cells at 0.1–1 fJ/search [19], [20], [44]), at  $K = 1.3 \times 10^5$ , 2,048-bit rows, per token per layer. TBD entries require the measurements of Sec. VII-F.

| Metric                           | CAM                     | GPU                  | Assumptions                                                |
|----------------------------------|-------------------------|----------------------|------------------------------------------------------------|
| Energy / search                  | 0.03–0.3 $\mu$ J<br>EST | ~67 mJ<br>EST        | cell figure spans published range; GPU is DRAM access only |
| Latency / search                 | 0.3–4.4 ns/arr.<br>EST  | 0.63 ms<br>EST       | reduction net. open; GPU is the batch-1 streaming floor    |
| Area / $10^3$ rows               | TBD                     | n/a                  | cell type (Table I), node                                  |
| Coverage / mm <sup>2</sup> , / W | TBD                     | TBD                  | behaviors $\times$ layers, at fixed detection rate         |
| Traffic / token                  | $dbL_{\text{mon}}$      | $K dbL_{\text{mon}}$ | measured here; Eqs. (2), (3)                               |
| Bank capacity                    | Fig. 6                  |                      | measured here; code widths from Sec. V                     |

says in bandwidth terms.<sup>9</sup>

Latency divides the same way. Streaming 2.1 GB at an H100’s  $3.35 \text{ TB s}^{-1}$  is a 0.63 ms floor per token per layer at batch 1, or roughly 10  $\mu$ s per request amortized at batch 64. Fabricated and modeled TCAM arrays report search times of 0.3–4.4 ns [20], [44]; with subarrays searched in parallel and mismatch counts reduced in a tree, a whole-bank search plausibly completes in tens of nanoseconds, and even at 100 ns it sits two orders under the amortized floor and roughly four under the batch-1 floor. Table VI carries these entries, marked as estimates, alongside the rows this paper measures and the rows that remain open.

#### F. The measurements that would replace the estimates

Each estimate above maps to a specific, short measurement, and the tooling for the array side already exists in-house. **(1) Array-level characterization with Eva-CAM:** Eva-CAM is a circuit/architecture-level evaluation tool for TCAM, MCAM and ACAM arrays across NVM technologies, validated against fabricated chips to within 6–20% [44]. Running it at a stated node for exactly the configurations this paper measures – 384- and 2,048-bit rows, banks of  $10^3$ – $1.3 \times 10^5$  rows, BCAM, TCAM and 3-bit MCAM cells – would replace the EST entries of Table VI with modeled energy, latency and area, including the subarray split and the reduction network the footnote above leaves open. This is days of work with an existing tool. **(2) A tuned GPU baseline:** a cuBLAS GEMV or FAISS scan [33] over the same banks on an H100 and a Jetson-class part,

<sup>9</sup>What the estimate omits, on each side: the GPU figure excludes the distance arithmetic itself and any cache reuse across batched requests (Sec. VII-F); the CAM figure takes array-level published values, which include matchline and sense-amp energy but not the query projection ( $\sim 10^6$  multiply-accumulates per token, negligible at this scale) or the mismatch-count reduction across subarrays. These are first-order literature values, not our measurements.

reported at batch 1 and batch 64 with measured power, so the amortization a datacenter part enjoys – and an edge part does not – appears in the table rather than in a caveat. Recall for any approximate index should additionally be measured under queries chosen *against* the index rather than drawn at random, since that is the regime a monitor operates in [45]. **(3) The two fidelity checks the encoding comparison still needs:** matched cell count for ternary vs. thermometer (Sec. V-B), and a common projection across the Hamming and Euclidean encodings on the readout workload, which settles whether the cell-type flip of Sec. VII-D is a device result or a projection artifact. Both are cluster runs on data already in hand. **[DO THIS: items (1) and (2) are the preemptive strike for a reviewer asking “is the CAM actually cheaper”: the paper now has the first-order numbers, and this list is the credible path to measured ones. If there is student time before the deadline, (1) is the highest value-per-day]**

## VIII. LIMITATIONS AND CONCLUSIONS

### A. Limitations

Several qualifications apply to everything above.

**What we have not built:** Nothing here competes with a deployed classifier, and no end-to-end jailbreak-blocking experiment was run. The measured claims concern the fidelity and cost of stored-signature matching, which is one component of the always-on stage described in Sec. I.

**The label:** The experiments define deception as an output that contradicts the known correct output. Deception in the stronger sense – a model representing a claim as false while asserting it – is harder to label, and the available datasets control for it imperfectly.

**Granularity:** The grid results classify one activation per response rather than per token, while the deployment target the hardware serves is per-token and always-on. The watchlist reproduction of Sec. VI-B is per-token, which is part of why it is reported here.

**Layer choice and model coverage:** Layer 33 is a tuned hyperparameter of one model family at one scale, and re-tuning per model is assumed. All grid results are one model. Re-tuning is also what makes the bank a rewritten structure, which is why endurance appears in Table V.

**Hardware numbers:** The comparison in Sec. VII is capacity and bandwidth arithmetic on vendor figures, with no contention term for a monitor co-resident with live inference. The energy and latency entries of Table VI are first-order estimates from literature values, not our measurements; Sec. VII-F lists what would replace them.

**Three metrics:** AUROC values in Sec. V are cross-domain on the ten-domain grid; the first pipeline’s numbers in Sec. IV-B are in-distribution on a four-domain grid under a different protocol; and the readout numbers in Sec. VI are F1 and top-1 agreement against a full-precision reference rather than against labels. None of the three is comparable with the others cell by cell, and the paper does not place them side by side.

**Digitized values:** the readout fidelity in Fig. 5 was digitized from result charts rather than read from result files, and the library-size sweep carries three stated implementation limitations (Sec. VI-D). Both should be regenerated from source before submission.

### B. Conclusions

Always-on behavioral monitoring of a language model is a library-search problem, and the variable that decides both its coverage and its cost is  $K$ , the number of stored signatures compared against each activation. At  $K = 1$ , which is what is deployed today, the operation is nearly free and no special hardware is warranted. Published work gives reasons to expect  $K$  to grow – non-transfer across behaviors, steering wanting a direction per behavior per domain per layer, feature dictionaries, and whole-vocabulary readout, which fixes  $K \approx 10^5$  per layer by construction – and also gives reasons to doubt it, which is why the question is worth measuring rather than asserting.

One methodological result comes before the fidelity numbers. Accuracy alone does not establish that a memory-based detector is using its memory: across five variants of an earlier pipeline, 66–100% of the accuracy of the two best-scoring configurations survived randomizing every stored row, so the trained front end was doing most of the classifying and the array was close to decoration. Reporting front-end share alongside AUROC separates the two, and it is what moved this work onto frozen projections.

What a CAM-resident monitor gives up can then be measured, and we measure it on the two workloads at opposite ends of the range. For deception detection at  $10^2$ – $10^3$  labeled rows, a 384-bit code inside a plain array holds in-domain AUROC within 0.009 of full-precision cosine similarity on the raw 8,192 numbers, at 0.920 (sign-RP ternary) and 0.919 (thermometer) against 0.928; cross-domain, the same codes give up 0.065 and 0.127. At a matched 384 bits the ternary code is ahead of the thermometer code by 0.061 on average, which is the opposite of what a preference for the cheaper cell would predict and is the experiment we would run next at matched cell count rather than matched code length. A two-stage shortlist recovers most of the cross-domain gap in aggregate, but its fixed  $k$  does not travel across bank sizes, which argues for a threshold criterion over a top- $k$  one in any array that has to serve banks of different sizes.

For whole-vocabulary readout at  $10^5$  rows per layer by construction, where the exact readout is its own ground truth and no labels are needed, a 3-bit Euclidean code at  $\sim 10^3$  storage bits per row – a  $128\times$  compression of an fp16 row – reproduces 0.96 of the exact top-1 selections and 0.81 of the per-row firing decisions. Reporting the closest row survives compression better than reporting which rows cross a threshold, by 0.15–0.2 at every code width, which is a matchline-mode result rather than a cell-type one. Swept over library size, the bank stays alive to  $10^4$  rows while per-row fidelity falls from 0.755 to 0.43 – and falls faster in full

precision than at 3 bits, so what gets harder with  $K$  is the discrimination rather than the arithmetic.

On the cost side, monitoring traffic on a GPU crosses an H100's entire HBM bandwidth at  $K \approx 204K$  and a Jetson-class part's at  $K \approx 17K$ , while the same library at the measured code widths is 6–33 MB per layer and fits on a die. A first-order estimate from literature figures puts the energy gap of the search itself at five to six orders of magnitude – roughly 67 mJ per token per layer to stream the whole-vocabulary bank from HBM against 0.03–0.3  $\mu J$  to compare it in place – of which a factor of 64 is the measured compression and the rest is the substrate (Sec. VII-E). A library that is inexpensive to hold and expensive to move is the condition under which searching it in place is worth the silicon. The remaining work is short and specific: Eva-CAM characterization at a stated node and a tuned GPU baseline to turn the estimates into measurements (Sec. VII-F), and – upstream of any hardware – establishing the minimum  $K$  that adequate behavioral coverage actually requires.

Things I would add before submission, in order: (1) the readout fidelity numbers regenerated from result files rather than digitized, and the library-size sweep re-run with the three fixes; (2) the matched-cell-count ternary vs. thermometer experiment; (3) the EST entries of Table VI replaced with Eva-CAM output and a measured GPU baseline; (4) the full 384 vs. 2,048-bit grids as figure panels; (5) a second model, even a small one, so the deception grid is not single-model. Item 1 is the one a reviewer could actually catch us on.

## REFERENCES

- [1] METR, “Common elements of frontier safety policies,” METR, 2026. [Online]. Available: <https://metr.org/common-elements>
- [2] Anthropic, “Constitutional classifiers++: Efficient production-grade defenses against universal jailbreaks,” *arXiv preprint arXiv:2601.04603*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.04603>
- [3] —, “Next-generation constitutional classifiers,” Anthropic research, 2026. [Online]. Available: <https://www.anthropic.com/research/next-generation-constitutional-classifiers>
- [4] Google DeepMind, “Building production-ready probes for gemini,” *arXiv preprint arXiv:2601.11516*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.11516>
- [5] UK AI Security Institute, “Our evaluation of claude mythos preview’s cyber capabilities,” AISI, 2026. [Online]. Available: <https://www.aisi.gov.uk/blog/our-evaluation-of-claude-mythos-previews-cyber-capabilities>
- [6] —, “Incident report: Unsourced agent behavior during cyber testing,” AISI, 2026, covering 25–28 July 2026. [Online]. Available: <https://www.aisi.gov.uk/blog/incident-report-unsourced-agent-behaviour-during-cyber-testing>
- [7] J. Clark, “Import AI 471: Why Hugging Face worries me; space mining; Five Eyes on AI,” Import AI, 2026, august 31, 2026. [Online]. Available: <https://importai.substack.com/p/import-ai-471-why-hugging-face-worries>
- [8] A. Cotra, “The Hugging Face attack surprised me,” Planned Obsolescence, 2026. [Online]. Available: <https://www.planned-obsolence.org/p/the-hugging-face-attack-surprised>
- [9] S. Marks and M. Tegmark, “The geometry of truth: Emergent linear structure in large language model representations of true/false datasets,” *arXiv preprint arXiv:2310.06824*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.06824>
- [10] C. Burns, H. Ye, D. Klein, and J. Steinhardt, “Discovering latent knowledge in language models without supervision,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [11] A. Zou *et al.*, “Representation engineering: A top-down approach to ai transparency,” *arXiv preprint arXiv:2310.01405*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.01405>
- [12] Z. J. Ying, S. Ravfogel, N. Kriesegkorte, and P. Hase, “The truthfulness spectrum hypothesis,” *arXiv preprint arXiv:2602.20273*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.20273>
- [13] T.-F. Berger, “Probing the limits of the lie detector approach to LLM deception,” *arXiv preprint arXiv:2603.10003*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.10003>
- [14] Schmidt Sciences, “AI interpretability program: Detecting deceptive reasoning in LLMs,” Schmidt Sciences, 2026. [Online]. Available: <https://www.schmidtsciences.org/ai-interpretability/>
- [15] Anthropic, “A global workspace in language models,” Anthropic research, 2026, july 6, 2026. [Online]. Available: <https://www.anthropic.com/research/global-workspace>
- [16] A. Templeton *et al.*, “Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet,” *Transformer Circuits Thread*, 2024. [Online]. Available: <https://transformer-circuits.pub/2024/scaling-monosemanticity/>
- [17] L. Gao *et al.*, “Scaling and evaluating sparse autoencoders,” *arXiv preprint arXiv:2406.04093*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.04093>
- [18] T. Lieberum *et al.*, “Gemma scope: Open sparse autoencoders everywhere all at once on gemma 2,” *arXiv preprint arXiv:2408.05147*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.05147>
- [19] K. Pagiamtzis and A. Sheikholeslami, “Content-addressable memory (CAM) circuits and architectures: A tutorial and survey,” *IEEE Journal of Solid-State Circuits*, vol. 41, no. 3, pp. 712–727, 2006.
- [20] K. Ni *et al.*, “Ferroelectric ternary content-addressable memory for one-shot learning,” *Nature Electronics*, vol. 2, pp. 521–529, 2019.
- [21] A. Kazemi *et al.*, “In-memory nearest neighbor search with FeFET multi-bit content-addressable memories,” in *Design, Automation and Test in Europe (DATE)*, 2021. [Online]. Available: <https://arxiv.org/abs/2011.07095>
- [22] C. Li *et al.*, “Analog content-addressable memories with memristors,” *Nature Communications*, vol. 11, no. 1, p. 1638, 2020.
- [23] A. Kazemi, F. Müller, M. M. Sharifi, H. Errahmouni, G. Gerlach, T. Kämpfe, M. Imani, X. S. Hu, and M. Niemier, “Achieving software-equivalent accuracy for hyperdimensional computing with ferroelectric-based in-memory computing,” *Scientific Reports*, vol. 12, p. 19201, 2022.
- [24] S. Dutta *et al.*, “Monolithic 3D integration of high endurance multi-bit ferroelectric FET for accelerating compute-in-memory,” in *IEEE International Electron Devices Meeting (IEDM)*, 2020.
- [25] K.-H. Kim *et al.*, “Scalable CMOS back-end-of-line-compatible AlScN/two-dimensional channel ferroelectric field-effect transistors,” *Nature Nanotechnology*, vol. 18, 2023.
- [26] GSI Technology, “Gemini apu: Enabling high-performance billion-scale similarity search,” GSI Technology, 2025. [Online]. Available: <https://gsitechnology.com/gemini-apu-enabling-high-performance-billion-scale-similarity-search/>
- [27] DeepSeek-AI, “DeepSeek-V3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024, 671B-parameter MoE;  $d = 7,168$ , 61 layers. [Online]. Available: <https://arxiv.org/abs/2412.19437>
- [28] M. Sharma *et al.*, “Constitutional classifiers: Defending against universal jailbreaks,” *arXiv preprint arXiv:2501.18837*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.18837>
- [29] Anthropic Alignment Science, “Cheap monitors,” Anthropic alignment blog, 2025. [Online]. Available: <https://alignment.anthropic.com/2025/cheap-monitors/>
- [30] P. Indyk and R. Motwani, “Approximate nearest neighbors: Towards removing the curse of dimensionality,” in *ACM Symposium on Theory of Computing (STOC)*, 1998, pp. 604–613.
- [31] M. S. Charikar, “Similarity estimation techniques from rounding algorithms,” in *ACM Symposium on Theory of Computing (STOC)*, 2002, pp. 380–388.
- [32] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [33] Meta Engineering and NVIDIA, “Accelerating gpu indexes in faiss with nvidia cuvs,” Meta Engineering blog, 2025. [Online]. Available: <https://engineering.fb.com/2025/05/08/data-infrastructure/accelerating-gpu-indexes-in-faiss-with-nvidia-cuvs/>

- [34] NVIDIA, “NVIDIA H100 tensor core GPU datasheet,” NVIDIA, 2023, 3.35 TB/s HBM3, 50 MB L2. [Online]. Available: <https://resources.nvidia.com/en-us-tensor-core>
- [35] Lyceum Technology, “LLM inference tokens per second: 2026 benchmarks,” Online benchmark roundup, 2026. [Online]. Available: <https://lyceum.technology/magazine/llm-inference-tokens-per-second-comparison-2026/>
- [36] W. Jiang, J. Clemons, K. Sankaralingam, and C. Kozyrakis, “How fast can I run my VLA? demystifying VLA inference performance with VLA-perf,” *arXiv preprint arXiv:2602.18397*, 2026, nVIDIA Research;  $\pi_0$  at 19 Hz on Jetson Thor, 50-action chunks. [Online]. Available: <https://arxiv.org/abs/2602.18397>
- [37] S. Ravfogel, Y. Elazar, H. Gonen, M. Twiton, and Y. Goldberg, “Null it out: Guarding protected attributes by iterative nullspace projection,” in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
- [38] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, “Learned step size quantization,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [39] Apple, “Apple intelligence foundation language models,” *arXiv preprint arXiv:2507.13575*, 2025. [Online]. Available: <https://arxiv.org/abs/2507.13575>
- [40] “A dual-mode 1FeFET-1C embedded memory: Characterization, eDRAM benchmarking, and a ternary CAM proof of concept,” 2026, manuscript in preparation.
- [41] Llama Team, AI @ Meta, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [42] NVIDIA, “Jetson AGX Thor developer kit,” NVIDIA product documentation, 2025, 273 GB/s LPDDR5X, 128 GB, 130 W. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-thor>
- [43] M. O’Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, “Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems,” in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, hBM2 access energy  $\approx 3.97$  pJ/bit.
- [44] L. Liu, M. M. Sharifi, R. Rajaei, A. Kazemi, K. Ni, X. Yin, M. Niemier, and X. S. Hu, “Eva-CAM: A circuit/architecture-level evaluation tool for general content addressable memories,” in *Design, Automation and Test in Europe (DATE)*, 2022.
- [45] A. Andoni, T. Haris, E. Kelman, and K. Onak, “Efficient algorithms for adversarially robust approximate nearest neighbor search,” *arXiv preprint arXiv:2601.00272*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.00272>

**Round-2 markup pass (Sep 2), from the round-2 log on the v05 print (v05 → v06):** (1) *Blue, verbatim:* the “Interpretability work that sets  $K$ ” paragraph deleted (its cites survive in Sec. I and the appendix); the Sec. V small- $K$  paragraph deleted; “From a hardware perspective,”; Sec. V’s opener carries your “a”/“workload” inserts – your strikes on “This section”/“evaluates” left no subject or verb, so it reads “We first evaluate,” from your “We 1st” cue (flagged in place). (2) *Sec. IV reframe, per your (B) thread:* intro rewritten generically around mapping activation signatures onto CAM arrays, spanning emerging devices to nearer-term CMOS, with your NVM parenthetical and the reference-point sentiment kept; IV-B leads with what FeFETCAMs could allow (richer representation or fewer cells; down-projection needed regardless); old IV-C cut to a transition per your call in chat, old text in a source comment; encoding headings updated (“or via other NVMs” / plain binary CMOS). (3) *Simplification thread:* Encoding 1’s opener de-AI’d, the hyperplane explanation shortened, the “contribution here” sentence deleted. (4) *Written:* the measured-FeFET-CAM mention by Table I and the subarray-split citation, both from the Scientific Reports paper you attached (kazemi2022hdcfefet); the 1FeFET-1C ternary-CAM paragraph with plausible extensions in the thermometer section (fefet1c2026 – authors TBD, flagged); the how-to-read-the- grids sentence (your (E), kept to one sentence per your note that the cross-domain paragraph covers part of it); the in-domain fidelity paragraph in plain English, numbers unchanged; property (3)’s  $K$ -scores clause is now a footnote per your call in chat. (5) *Left for people, not prose:* the decision-rule question carries an orange flag – routed to Indranil (the round-2 log’s “Ismail” was a misread of MN’s margin note); the Truthfulness↔VLA framing and the Fig. 2(a)–(c) keep/cut call are violet-flagged for conversation (my read on the middle path – keep only (b) – is in the note); the DAC-thermosensitive citation has an orange flag awaiting the PDF from your work machine.

**Fourth hand-markup pass (Sep 1), from the pass-2 markup log and your chat clarifications (v04 → v05):** (1) *Blue, verbatim:* “ $K = 1$  represents,” “are potentially valuable,” “also have no batch,” “still must move” (your “a must” = insert *must*) + “Rather, it is better to consider,” “(2) A per-token price,” “here, the  $K \approx 10^2$ – $10^3$  regime”; your hedged “reinforce this point” is applied with a violet veto note. (2) *Comment (D), as the one move you suggested:* “What ‘CAM-resident’ requires” now leads Sec. IV after a reframed intro (technology-enabled first, CMOS as the nearer-term study, your “can or cannot buy” framing); encodings 1–3 are presented as options against the requirements, thermometer flagged as the plain CMOS route, shortlist+rerank reframed as checking a smaller  $K$  a GPU serves easily (with the retrieval citation). The confusing closing sentence (“the route back”) is rewritten and flagged. (3) *Written for you:* the abstract’s emerging-technology clause; the OpenAI–Hugging Face augmentation in the intro (from IV.pdf; the pen’s “Anthropic”

read as OpenAI – two new refs, Clark and Cotra) with a conservative-arithmetic clause on the  $10^3$  rung; the II-B echo of the associative-search paragraph (your (C)); the decision rule now says multiple exemplars per class and carries your few-shot analogy; V-A is tutorial and introduces layer 33 properly (inherited from the source study’s tuning), with IV-B pointing there; the pre-emptive “why small  $K$ ” remark ends the Sec. V intro. (4) *Cuts, content preserved:* shared-circuitry pointers (II-C, III) → red-team objection (1); ANN block → two sentences + objection (2); footnote 4 deleted; IV’s experiment list → one audit sentence in V-E; V-B recap cut. (5) *Your Fig. 2 question, answered in place:* their Fig. 2 trains on seven datasets and tests on ten (three on-policy benchmarks are test-only); our exemplar banks cover all ten, so our richer grid is expected – violet note at the figure discussion, and the V-A naming flag now includes ethics/ethical. The combined-data check for panels (d)–(f) stays queued for I. Bera. (6) *Flagged for your read, not blockers:* “reinforce this point” (veto note), the rewritten route-back sentence, “384-bit-equivalent” + the 128-cells-per-matchline reading, and the “their limits” reading in the small- $K$  remark.

**Bandwidth-tax appendix and red-team addition (Aug 28 pm), per chat:** (1) New appendix (file 10, Sec. X): what a 10–20% bandwidth tax does to inference on an H100 – first-order 1:1 pass-through in decode with the batch-1 and batch-32 arithmetic, the three second-order effects (queueing amplification to fleet size, L2/DRAM contention overshoot, proportional energy), and the five thief/MIG/end-to-end/SLO experiments, all days-scale. Parked as an appendix per your note; easy to move into Sec. VII or cut. (2) Red-team objection (1) now carries your saturation argument: the full-precision ceiling itself transfers poorly (0.705 vs. 0.928), and wider codes make cross-domain transfer worse, so capacity is not the limiter – consistent with behaviors wanting their own stored directions, i.e., with  $K$  growing. Hedged to one model/one grid, with the shared-subspace counter-reading left standing.

**Second hand-markup pass (Aug 28 pm): Sec. II-C and the rest of Sec. III:** (1) *Blue, verbatim:* “ $L$ -d has not changed much”; “An important ‘stress test’ for library size is cross-domain – i.e.,”; “Two vocabulary items recur below” deleted; “that will be reproduced here.” (2) *Pink, written:* the in-line  $L$ - $d$  example (671B MoE at  $d=7,168 \times 61$  layers vs. dense 70B at  $8,192 \times 80$ ), which also supplies the citation you asked for (DeepSeek-V3 technical report, added to refs.bib – give it a look). (3) *Orange:* the shared-circuitry counter-argument sentence and the whole ANN block are now red in both builds, tracked for likely cutting; the ANN block also picks up “i.e., a monitor that cannot afford to miss.” On your “can we test this”: it is the adversarial-recall measurement already queued as item (2) of the Sec. VII measurement plan, noted in violet in place. (4) *Sec. III restructure, per your boxes and our exchange:* Sec. III is now the very short “A Cost Model with One Variable” – equations, crossings, workload anchors, the per-token/compounding argument, and the session-length point. Cut: old III-A (redundant with the intro ladder), the dashed-lines paragraph, and Table III (your

“seems redundant”). Folded into the intro: 122 runs across seven models. The old subsection labels survive as aliases so cross-references resolve. This recovers roughly a page toward the six-page target.

**Hand-markup pass (Aug 28), from your pen on the printed v04 (abstract, Sec. I, Sec. II-A, part of Sec. III; Sec. II-C untouched per your note):** (1) *Blue, applied verbatim*: the abstract parentheticals, “aforementioned defense,” “for a whole-vocabulary readout,” “On a GPU,” “so if,” the CAM sentence rewrite, and the (1)/(2) workload numbering; in Sec. I the period/“Only” edit, “All of said outcomes were found retrospectively,” “(random chance),” the ladder rung rewrites (“captures,” “is indicative of the above campaign,” “– i.e.,” twice, “is representative of,” “and it is”), “Finally,  $K$  could grow even higher...two orders of magnitude higher than the J-space,” “That said, we cross...well before this. Per Fig. 1(b),” and “Our contributions include”; in Sec. II-A the “Two choices” sentence and both bold lead-ins deleted, “and can measure Hamming distance,” and “With appropriate peripheral circuitry we can measure.” Two verbatim applications carry violet notes where the marked wording leaves a gap (the abstract’s “in memory...in the memory array itself” doubling; the verbless “Only flagged exchanges escalated”). (2) *Pink, written for you*: the opener is now “Recent security evaluations show why such a monitor is needed”; “cascade’s” is “defense’s”; the 22-of-32 sentence carries the two-thirds reading; the clean-text sentence now says each message can read as routine while the sequence is an attack; the lie-detector sentence is “miss deception carried by technically true statements”; the highlighted  $K$ /coverage sentence is rewritten (each behavior–domain–layer combination is one more stored row); and the CAM-proposal paragraph now names near-term SRAM CAMs and emerging-device CAMs. (3) *The new panel*: Fig. 1 is now (a)+(b) – (b) is the bandwidth-consumption plot you sketched on Table II (limiting  $K$  vs token rate, 100% lines for H100/A100/Thor, 10%/1% H100 guides, the whole-vocabulary point marked). The old Table II is retired; its 25-layer numbers survive in the Sec. III-B text. On your batch question: the token rate is the part’s aggregate rate, so batching moves a deployment along the axis rather than adding one – said in the caption, with per-request amortization left to red-team objection (3). (4) *Orange*: the spider example is a footnote; the four cell names and exact/threshold are bold (*best* left italic – flagged in place for your call).

**Intro expansion (Aug 26 pm), per your note that readers should quickly care**: the intro gains roughly half a column in three beats. (1) The security problem: the long-autonomous-operation framing ( $10^8$ -token runs, 22 of 32 steps), the four unsanctioned-action categories from the AISI campaign stated in full, and that all of it was found retrospectively with nothing watching internal state – clean text, unclean operation. (2) The deception work: probes recover truth-related structure but the directions are narrow (chance on sycophancy; non-lying deception missed; the Schmidt program), which is what multiplies  $K$ . (3) J-space: the rung now carries the spider example and the staged-safety-test catch, so the reader meets

the workload before Sec. VI; Sec. VI-A points back at the example instead of repeating it. Abstract + intro is now roughly 2.3 columns; the two candidate cuts from the earlier accounting still stand if you want some of that back.

**What changed in v04 (Aug 26), against your new flags on 00–04 and your note that the abstract is essentially the intro:** (1) The abstract is culled to roughly half a column: the ladder compressed to one sentence, the  $10^3$ -rung examples, the 2.1 GB/1,600 tokens/s arithmetic, the maturity-ladder recitation, and the firing-F1 number all moved out (each survives in the intro or Secs. III/VI–VII). Your “support this search” and “albeit with potentially lower” edits are kept. (2) The intro is rebuilt from the abstract at about one column: your STAR-A material (what the  $K = 1$  check stores, compares, escalates) folded in right after the opening, ending with the point that the probe is a small share even of the cascade’s 1–3.5%; the “related work suggests  $K$  is growing” framing is the transition paragraph, with the Schmidt reference kept to one clause; the ladder is one bold clause per rung, with the  $10^3$  rung carrying the four behavior categories and the domains  $\times$  layers arithmetic, and the  $1.3 \times 10^5$  rung now carrying the J-space lead-in you asked for (what the readout method does, with the citation). The probe cost arithmetic ( $2Ld$ , the MoE point), the  $23.7\% \rightarrow 1\text{--}3.5\%$  progression, and the reconciliation all moved to Sec. II-C. **Length accounting**: abstract + intro text lands at roughly 1.8 columns (abstract  $\sim 0.5$ , intro  $\sim 1.3$ ), against your “about 1.5 at most.” The remaining 0.3 column comes out of content you asked to add, so I stopped rather than choosing: the two candidate cuts are (a) the  $K \approx 10^7$  rung and the crossing sentence (both survive in Sec. III), or (b) the contribution clauses’ numbers. Say which and it is a five-minute change. (3) Sec. II is restructured per your flag: II-A the CAM design space, II-B a technology-implementations placeholder seeded with the BEOL material and marked for your content, II-C preliminaries (the probe/AUROC/domain-behavior material, trimmed) plus related work. (4) Sec. III-D (“What would keep the library small”) is cut from the body and expanded into a red-team appendix (new file 09), which also absorbs the ANN objection and the batch-amortization caveat, each stated with a response. The intro no longer carries the counter-evidence footnote. (5) The tok/s reality check is in Sec. III-B: reported aggregate serving throughputs for 70B-class models are of order  $10^3$  tokens/s per datacenter GPU at high batch, tens per interactive user, so the  $10\text{--}10^4$  bracket spans deployed practice (gray-literature benchmark cite, flagged for replacement if you have the interconnect number). [Aug 28: the old Table II is now retired in favor of Fig. 1(b), per your pink note.] (6) Sec. IV opens with a maturity-order lead-in (Euclidean ideal first, then nearer-term CMOS), and the audit-table caption now says it is the sanity check against “cheating.”

**Abstract pass (Aug 26), against your in-file edits and flags:** (1) Your edits are kept (the Anthropic-named opener, “target  $K$ ’s are rapidly increasing,” the softened transfer claim), with the stray punctuation cleaned up. (2) The  $K \approx 10^3$  rung no longer carries a date: it now names the application –

unsanctioned agent behaviors from cyber-security evaluations, with two concrete examples – and states the behavior  $\times$  domain  $\times$  layer multiplication. (3) “Consume a deployed part” is now “the monitor’s memory traffic alone can exceed the bandwidth of the part it runs on.” (4) The CAM is introduced as a proposal first (“We propose to perform this search with CAMs”), then framed per your note: the distance function is computed in the memory array itself, so the streaming traffic and the GPU bandwidth loss are eliminated, albeit at lower precision and with potentially lower-fidelity distance functions. (5) The 0.920/0.919/0.928 recitation is unpacked: the 0.928 is named as the full-precision cosine ceiling, the two codes are stated against it, and the sentence now says what the compression ratio is ( $>300\times$  per signature). J-space is named in the abstract and its readout result now includes the firing-F1 number (0.81) alongside top-1 (0.96). (6) One inconsistency fixed: the abstract said 3–5 orders of magnitude for the energy gap; Sec. VII’s estimate is five to six, and the abstract now matches.

**What changed in v03 (Aug 25), against your markup on 00 and 01 and the follow-up requests:** (1) Title is option (2) from your note, as you leaned. (2) The  $K$  ladder is explicit and bold –  $K = 1$ ,  $K \approx 10^2$ ,  $10^3$ ,  $1.3 \times 10^5$ ,  $10^7$  – in the abstract (compressed), Sec. I (full, one rung per sentence), and Sec. III (with the arithmetic). The  $10^7$  rung carries the “usable slice TBD” qualifier inline, per your note. (3) Figs. 1 and 2 are merged: fig\_ladder\_v1 keeps the trajectory figure’s look, adds a right-hand axis reading the same positions as GPU traffic in GB/s, the two bandwidth ceilings, and the flat CAM line. It sits in the intro as Fig. 1 and is referred to there. The old full-width traffic figure is retired from placement (still in gen\_paper\_figs.py), which recovers roughly a third of a page. (4) Sec. I now describes what the deployed  $K = 1$  check stores, compares, and escalates, in plainer language; the overhead progression is flipped (23.7%  $\rightarrow$  1–3.5%  $\rightarrow$   $\sim 0.001\%$ ) and the 1–3.5% vs.  $10^{-5}$  tension is reconciled in a footnote – the  $10^{-5}$  is the probe arithmetic alone, the 1–3.5% is the deployed cascade – with an orange DO-THIS flag to verify that reading against the papers. (5) Related work on deception detection is consolidated with two additions: the lie-detector-limits preprint (probes trained on lying miss non-lying deception) and the Schmidt Sciences interpretability program on detecting deceptive reasoning. Both entries are in refs.bib with URLs; they were found and checked Aug 25. (6) Sec. II now has a BEOL paragraph (Dutta IEDM 2020 monolithic 3D multi-bit FeFET; BEOL AlScN/2D-channel FeFETs) and Sec. VII’s placement table points at it. (7) Per your answer that the POC results are the AUROC data, the new quantitative cost case in Sec. VII-E is a first-order energy/latency estimate from literature figures – HBM at  $\sim 4$  pJ/bit (O’Connor, MICRO 2017) against CAM search at 0.1–1 fJ/cell (Ni 2019; Eva-CAM validation values) – presented with the sensitivity argument (a  $10\times$  error in the cell figure moves nothing qualitative) and a near-term measurement plan: Eva-CAM runs at a stated node for our exact row widths, a tuned GEMV/FAISS GPU baseline at

batch 1 and 64, and the matched-cell-count experiment. Eva-CAM is the group’s own DATE 2022 tool, so the runs are days, not months. (8) Contribution (2) is reframed as the maturity ladder; contribution (3) is prose and claims the two reproductions, per your note on the framing check.

**Open items carried over from v02, still open:**

**1. Matched cell count, ternary vs. thermometer.** Unchanged: at matched 384 bits ternary wins cross-domain, but a ternary cell may cost two bits of hardware, so the fair experiment is matched cell count. Still the single most useful add before submission.

**2. The 2,048-bit grids are summarized, not shown.** Unchanged; Sec. V reports the aggregate counts only.

**3. J-space numbers are digitized.** Unchanged: Fig. 7 panels were read off the 07-30-26 deck charts to  $\pm 0.02$ , and the library-size sweep carries its three stated limitations. Getting the arrays from John, or the re-run, is still the highest-value item.

**4. References.** Block 2 of refs.bib still needs its verification pass. New in v03 and checked against sources on Aug 25: oconnor2017fgdram, liu2022evacam, dutta2020m3dfefet, schmidt2026interp, berger2026liedetector. Added but needing an author-list check: kim2023alscn.

**5. Length.** v03 retires one full-width figure and the draft is still long. Next cuts, in order: Sec. II-C (related work), Sec. VI-C, Table V.

**6. Camera-ready.** Author order (I. Bera first?) and funding text still to settle; the draft stays anonymized.